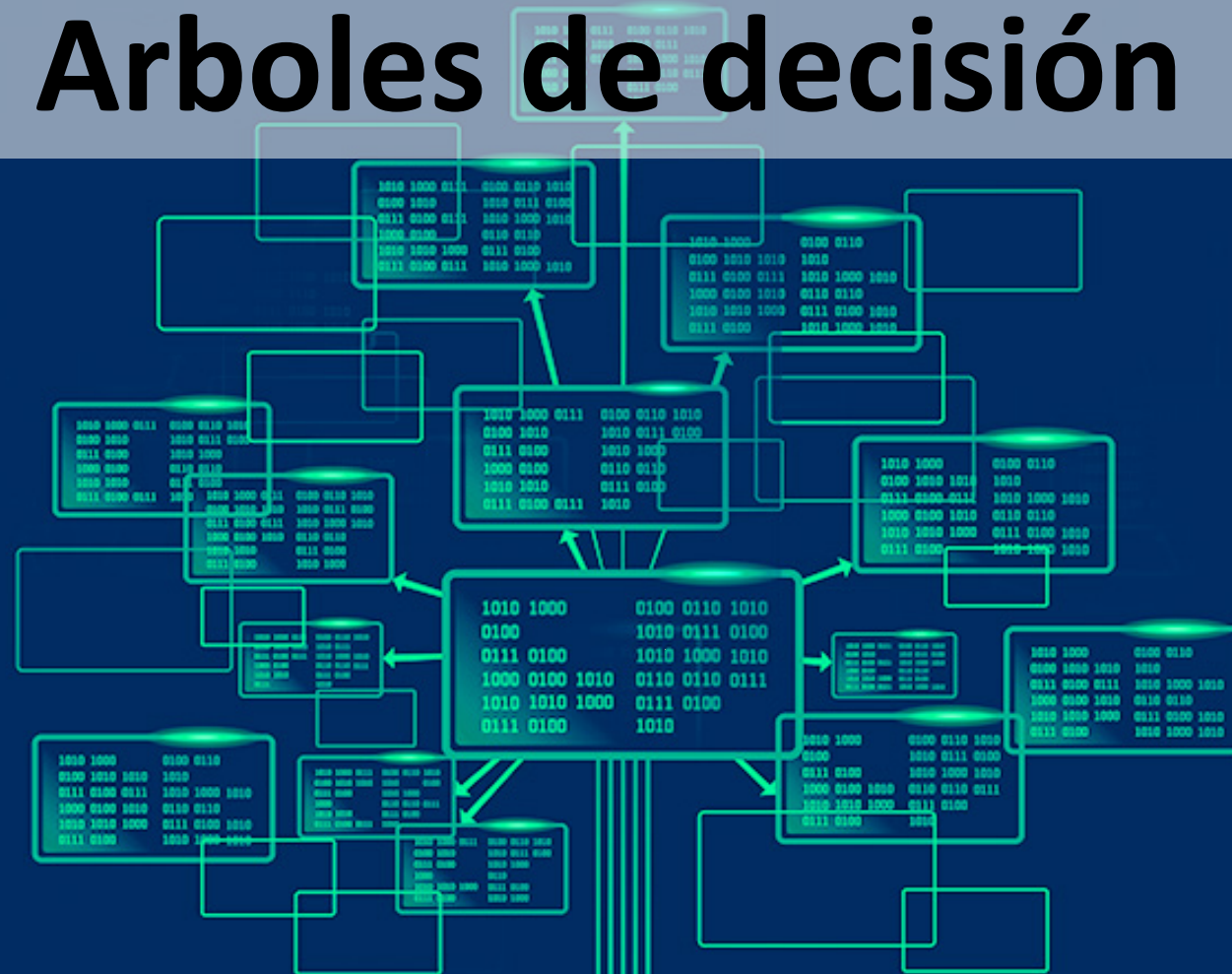


Arboles de decisión



2025

Flavio E. Spetale



Arboles de decisión

Son uno de los modelos de minería de datos más comunes y estudiados, y no precisamente por su capacidad predictiva, superada generalmente por otros modelos más complejos, sino por su alta **capacidad explicativa** y la **facilidad para interpretar** el modelo generado.

Un árbol de decisiones es un **modelo jerárquico** utilizado para respaldar las decisiones que describe las decisiones y sus resultados potenciales, incorporando eventos casuales, gastos de recursos y utilidad. La estructura de árbol se compone de un nodo raíz, ramas, nodos internos y nodos de hoja, formando una estructura jerárquica similar a un árbol.

El árbol de decisiones es una técnica de aprendizaje supervisado no paramétrico que se puede utilizar tanto para problemas de **clasificación** como de **regresión**, pero sobre todo se prefiere para resolver problemas de clasificación.

Tipos de árboles de decisión

El algoritmo de Hunt, 1960, se desarrollo para modelar el aprendizaje humano en Psicología.

Algoritmos de árboles de decisión populares:

1. **ID3** (Iterative Dichotomiser 3): Este algoritmo fue desarrollado por Ross Quinlan en 1986. Este algoritmo aprovecha la entropía y la ganancia de información como métricas para evaluar las divisiones de candidatos.
2. **C4.5**: Este algoritmo fue desarrollado por Ross Quinlan y se considera una evolución del ID3. Puede utilizar la ganancia de información o las proporciones de ganancia para evaluar los puntos de división dentro de los árboles de decisión.
3. **CART** (Classification And Regression Trees): Este algoritmo fue introducido por Leo Breiman. Generalmente utiliza la impureza de Gini para identificar el atributo ideal para la división. La impureza de Gini mide la frecuencia con la que se clasifica incorrectamente un atributo elegido al azar. Cuando se evalúa usando la impureza de Gini, un valor más bajo es más ideal.

Terminologías del árbol de decisión

Nodo raíz: el nodo raíz es desde donde comienza el árbol de decisión. Representa el conjunto de datos completo, que además se divide en dos o más conjuntos homogéneos.

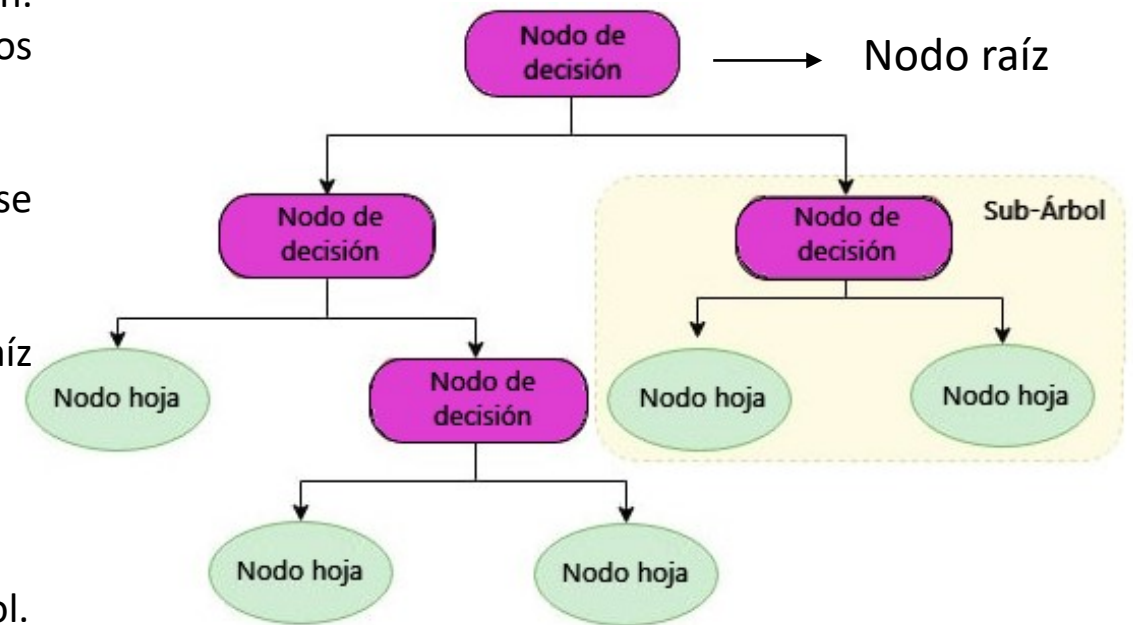
Nodo hoja: los nodos hoja son el nodo de salida final y el árbol no se puede segregar más después de obtener un nodo hoja.

División: la división es el proceso de dividir el nodo de decisión/nodo raíz en subnodos de acuerdo con las condiciones dadas.

Rama/Subárbol: Un árbol formado al dividir el árbol.

Poda: La poda es el proceso de eliminar las ramas no deseadas del árbol.

Nodo padre/hijo: el nodo raíz del árbol se denomina nodo padre y los demás nodos se denominan nodos hijos.

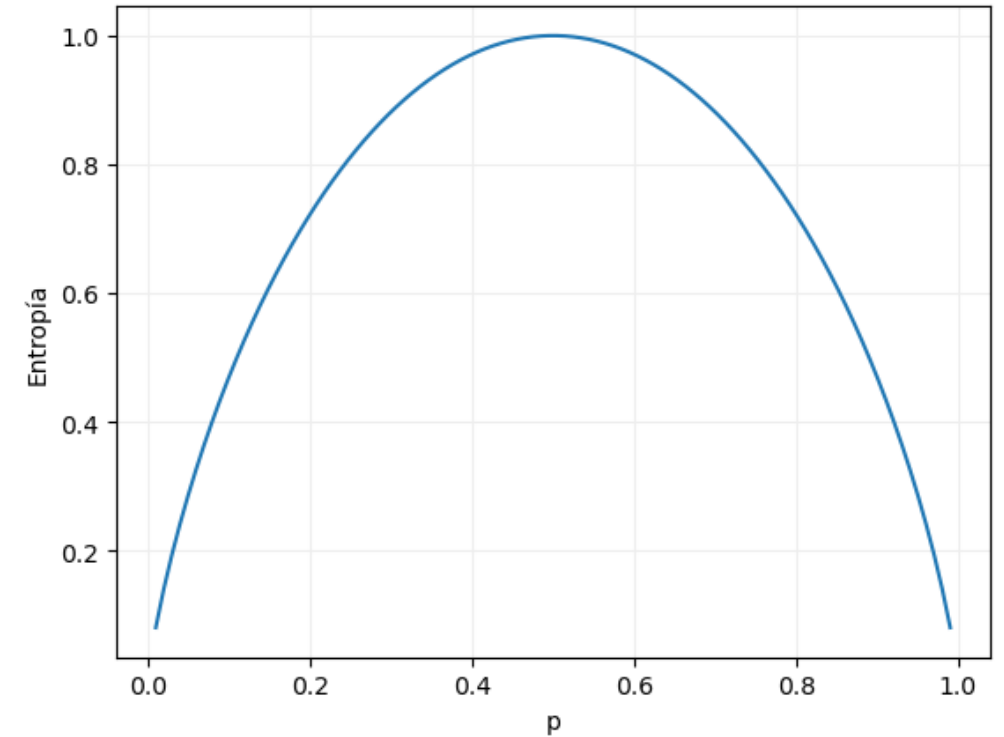


Medidas de Impureza

La construcción del árbol se basa en minimizar la impureza de los nodos resultantes después de cada división.

Cada p_i se calcula como el número de observaciones de la clase i presentes en la hoja t ($n_i(t)$) dividido por el total de observaciones en dicha hoja n_t .

$$p_i(t) = n_i(t) / n_t$$



Los valores de entropía pueden estar entre 0 y 1

Medidas de Impureza

La entropía (Criterio de Information Gain): es un concepto que se deriva de la teoría de la información, que mide la impureza de los valores de la muestra.

$$H(S) = - \sum_{i=1}^c p_i \cdot \log_2 p_i$$

S es el conjunto de datos, c es la cantidad de clases y p_i es la probabilidad de una clase

Índice de Gini: es la probabilidad de clasificar incorrectamente un punto de datos aleatorio en el conjunto de datos si se etiquetara en función de la distribución de clases del conjunto de datos

$$Gini = 1 - \sum_{i=1}^c p_i^2$$

p_i es la probabilidad de una clase, c es la cantidad de clases

Criterio de División

Para un atributo A que divide el conjunto S en subconjuntos $\{S_1, S_2, \dots, S_v\}$:

Ganancia de Información (Information Gain):

$$IG(S, A) = H(S) - \sum_{j=1}^v H(S_j) \cdot \frac{|S_j|}{|S|} = H(S) - H(S, A)$$

Relación de ganancia (Gain Ratio, para atributos con muchos valores):

$$GR(S, A) = \frac{IG(S, A)}{-\sum_{j=1}^v \log_2\left(\frac{|S_j|}{|S|}\right) \cdot \frac{|S_j|}{|S|}}$$

$H(S)$ es la entropía del conjunto S, $|S_j|$ es el cantidad del subconjunto j del atributo A, $|S|$ es la cantidad total de observaciones de un conjunto S, v es el conjunto de valores distintos de un atributo A, $H(S_j)$ es la entropía del subconjunto j para el atributo A y $H(S, A)$ es la entropía de un atributo A.

Construcción conceptual de un Árbol de Decisión

1. Punto de partida

Se parte de un conjunto de ejemplos (datos de entrenamiento) y de un conjunto de atributos (variables predictoras), con un atributo objetivo que queremos predecir (la clase o etiqueta).

2. Criterio de homogeneidad

Si todos los ejemplos que llegan a un nodo pertenecen a la **misma clase**, no hace falta seguir dividiendo: ese nodo se convierte en una **hoja** con esa clase.

Esto refleja la idea de **pureza máxima** en un nodo.

3. Ausencia de atributos discriminantes

Si ya no quedan atributos con los cuales dividir, pero todavía hay ejemplos de clases distintas, el nodo se convierte en una **hoja con la clase mayoritaria**.

Este paso evita seguir dividiendo cuando no tenemos información adicional para discriminar.

4. Selección del mejor atributo

Si existen atributos disponibles, se selecciona aquel que proporcione la **máxima ganancia de información (IG)** o alguna otra medida de calidad de la partición (como Gini o χ^2).

Este atributo se convierte en el **atributo de decisión** del nodo actual.

La idea es que cada división busque maximizar la reducción de incertidumbre en la clasificación.

Construcción conceptual de un Árbol de Decisión

5. Creación de ramas

Para cada posible valor del atributo seleccionado, se crea una **rama** que conecta el nodo actual con un subárbol. Los ejemplos se **dividen** según los valores del atributo.

6. Subárboles y casos vacíos

Si para un valor del atributo no existen ejemplos, se agrega una hoja con la **clase mayoritaria** en el nodo padre. Si hay ejemplos, se construye el subárbol de manera **recursiva**, aplicando el mismo proceso con los atributos restantes.

7. Resultado final

El árbol completo se forma repitiendo este proceso recursivo hasta que todas las ramas terminan en **nodos hoja**.

Cada hoja corresponde a una regla de decisión (un camino desde la raíz hasta la hoja), lo que convierte al árbol en un **modelo interpretable**.

Árbol de decisión: Subajuste

Sesgo (bias): error por simplificar demasiado el problema (ej: “todos los pacientes tienen la misma enfermedad”).

Varianza: error por depender demasiado de los datos de entrenamiento (ej: “cada paciente es un caso único sin reglas generales”).

El trade-off es la **tensión inevitable**: si reducimos sesgo, aumentamos varianza; si reducimos varianza, aumentamos sesgo.

Cuándo ocurre el subajuste:

Cuando el árbol es **demasiado simple** para capturar los patrones de los datos.

Señales típicas:

Alto error tanto en el **entrenamiento** como en el **test** → el modelo no logra distinguir adecuadamente entre clases.

Causa principal:

El modelo tiene **alto sesgo (bias)**: hace suposiciones demasiado fuertes y simplistas sobre la estructura de los datos.

Árbol de decisión: Sobreajuste

Cuándo ocurre el sobreajuste:

Cuando el árbol es **demasiado complejo**, creciendo hasta capturar incluso el **ruido o las particularidades irrelevantes** de los datos de entrenamiento.

Señales típicas:

Error muy bajo en **entrenamiento**, pero alto en **test** → el modelo no generaliza bien a nuevos datos.

Causa principal:

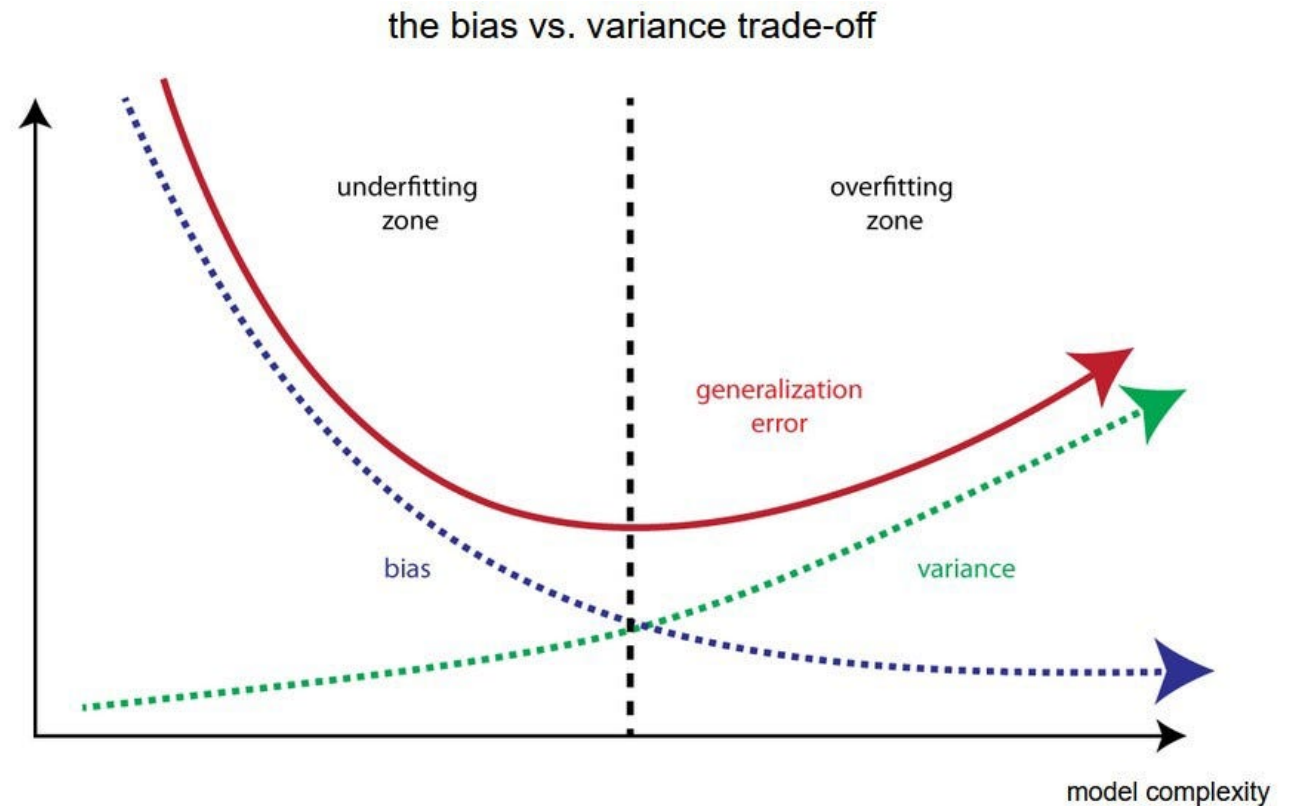
El modelo tiene **alta varianza**: se ajusta demasiado a los ejemplos concretos y pierde capacidad de generalización.

Trade-off entre sesgo y varianza

Encontrar el **punto de equilibrio** donde el modelo es lo suficientemente complejo para aprender patrones relevantes, pero no tanto como para aprender ruido.

En árboles de decisión, esto se maneja con:

- **Profundidad máxima** del árbol.
- **Número mínimo de ejemplos por hoja.**
- **Poda (pruning).**

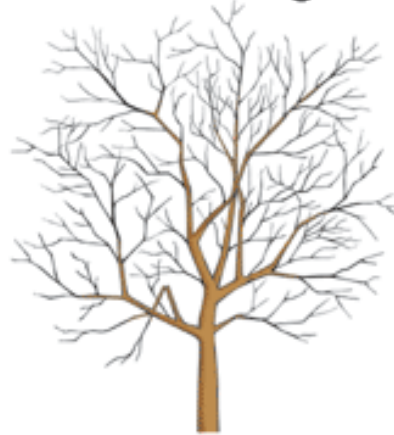


Árbol de decisión: Poda del árbol

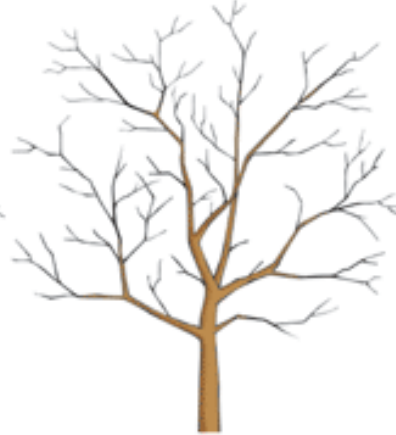
Consiste en eliminar aquellas hojas que son las que causan el *sobreajuste*. Para ello se calcula cuál es la partición (de la cual cuelgan las hojas a eliminar o, en general, un subárbol) que aporta menor ratio entre el incremento de profundidad media del árbol y el decremento del error global de clasificación.

Se eliminan aquellas hojas (o subárboles) que menos ayudan a mejorar el árbol durante el proceso de creación.

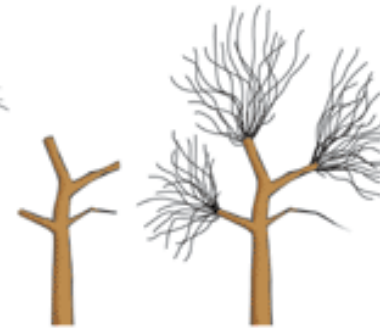
A Look at Pruning



GOOD

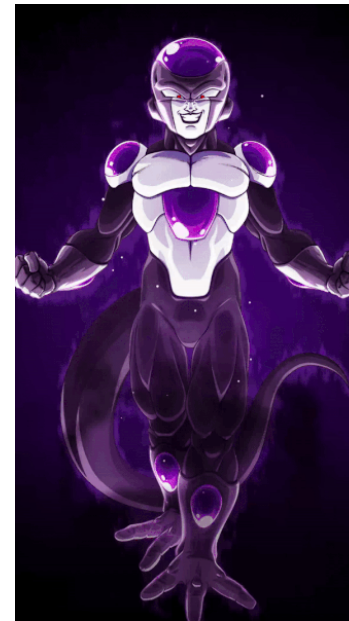


NOT GOOD



Árbol de decisión: Clasificación y CART

Queremos clasificar los personajes de Drago Ball Super. Cada uno de estos 20 personajes posee ciertas características, nivel de fuerza, defensa, costo de energía y letalidad cada una en una escala de 0 a 20. Para este ejemplo, solamente usaremos nivel de fuerza y letalidad.



Letalidad



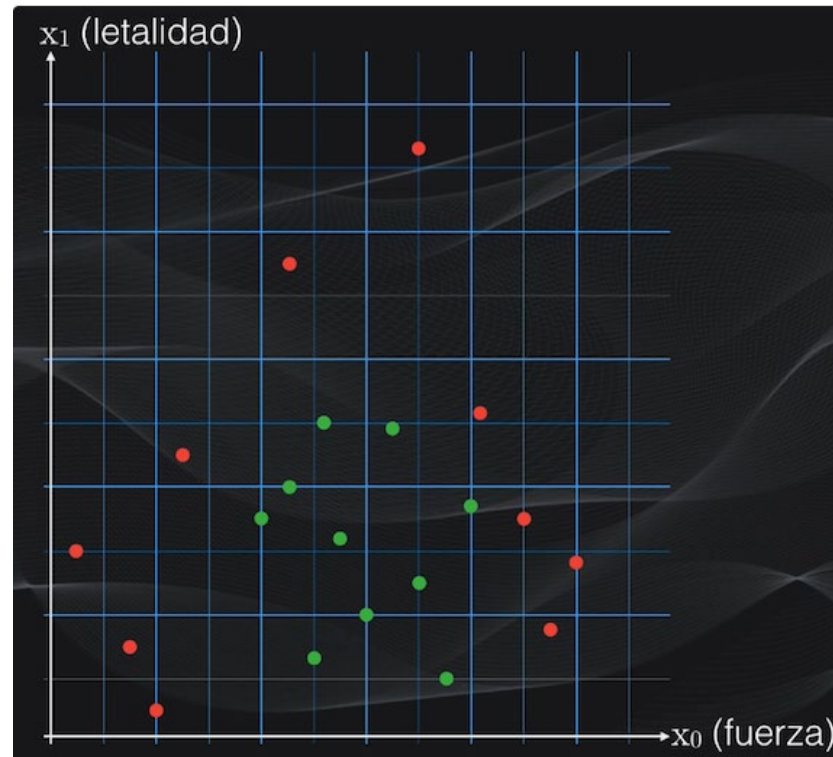
Fuerza



Árbol de decisión: Clasificación y CART

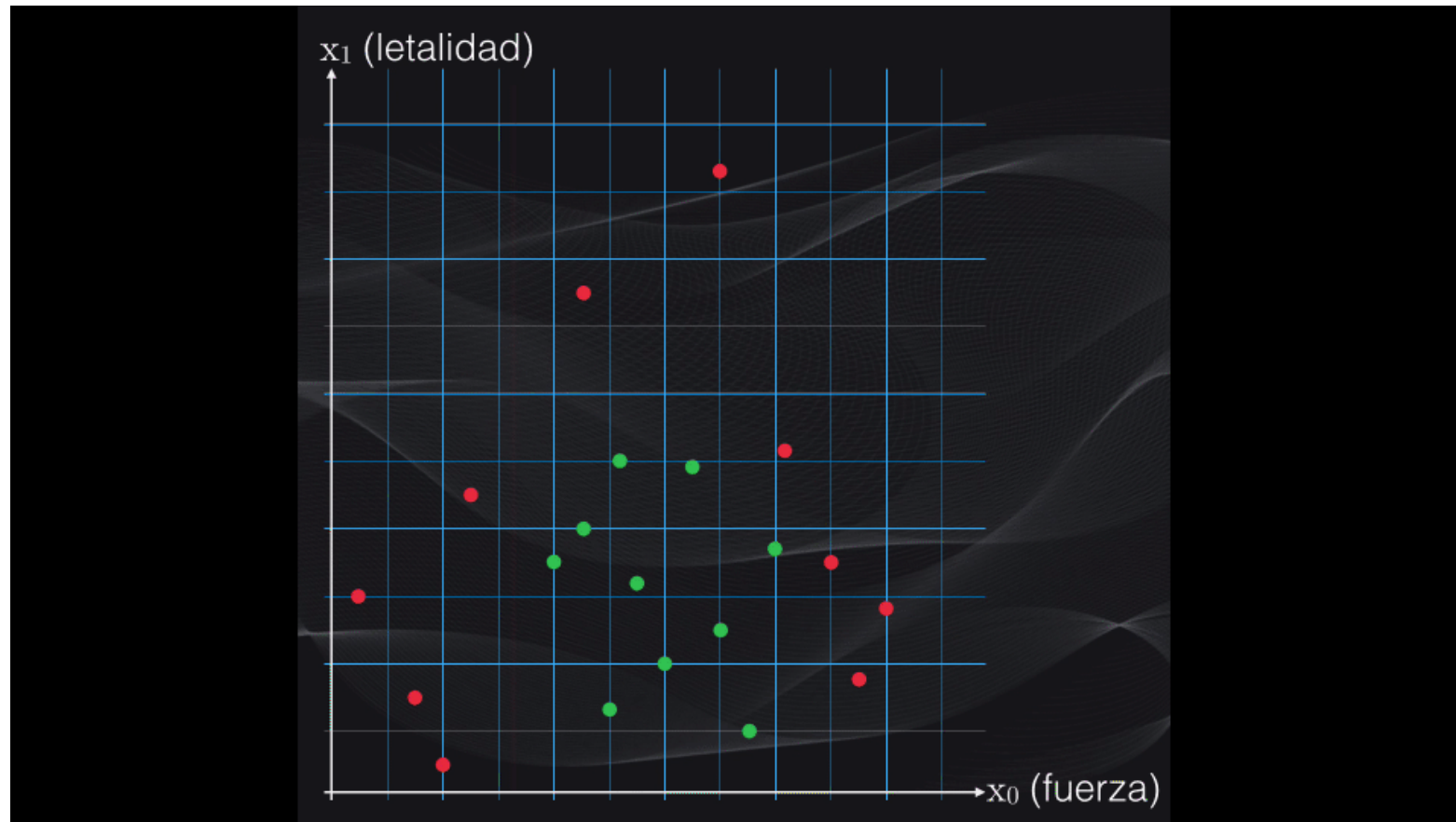
Los personajes malignos, 10, estarán representados con puntos rojos y los buenos, 10, con puntos verdes.

La idea es encontrar una forma de separar los personajes malignos de los buenos, i.e., la manera de calcular unas fronteras de decisión que permitan posteriormente clasificar nuevos personajes en una de estas dos categorías.



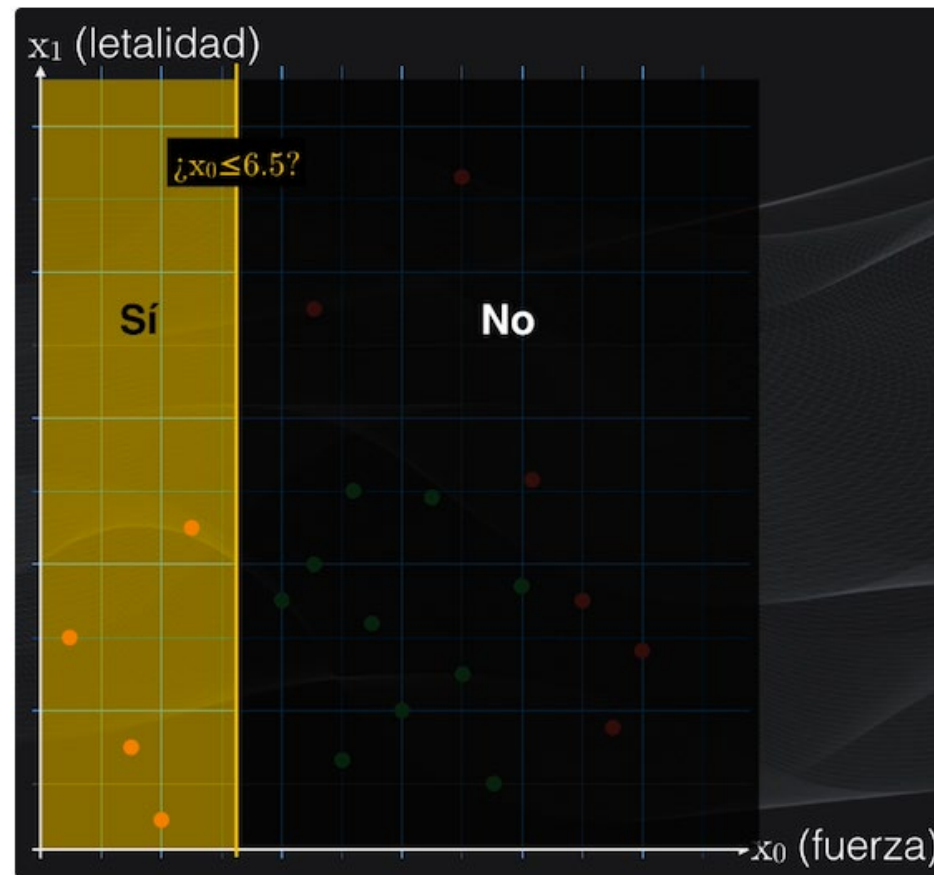
Árbol de decisión: Clasificación y CART

La idea de la clasificación con árboles de decisión es simple: iterativamente se irán generando particiones binarias (dos agrupaciones) sobre la región de interés, buscando que cada nueva partición genere un subgrupo de datos lo más homogéneo posible.



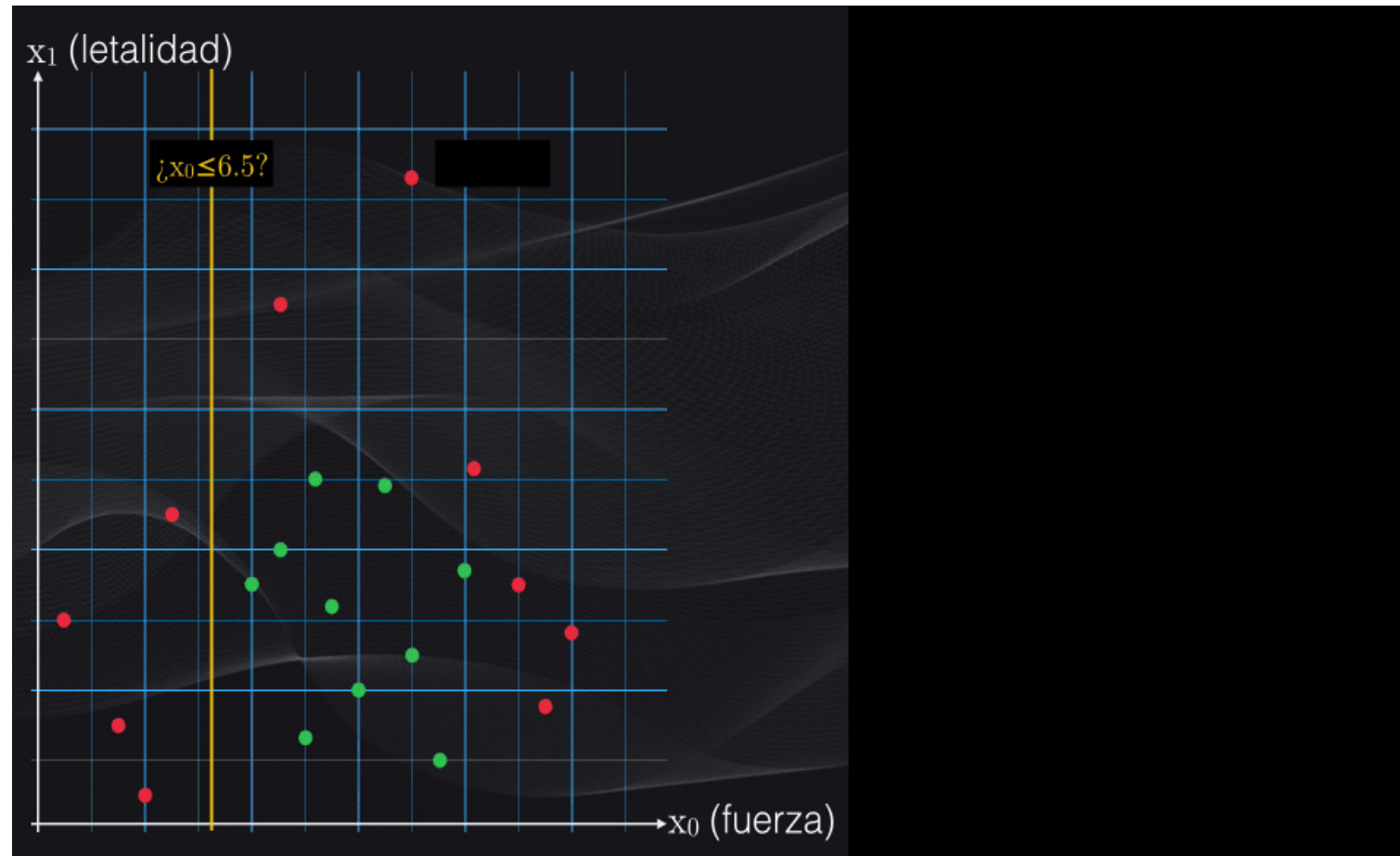
Árbol de decisión: Clasificación y CART

Primero se establece una condición. Dependiendo de si los datos cumplen o no la condición tendremos una primera partición en dos subregiones (de ahí el término particiones binarias).



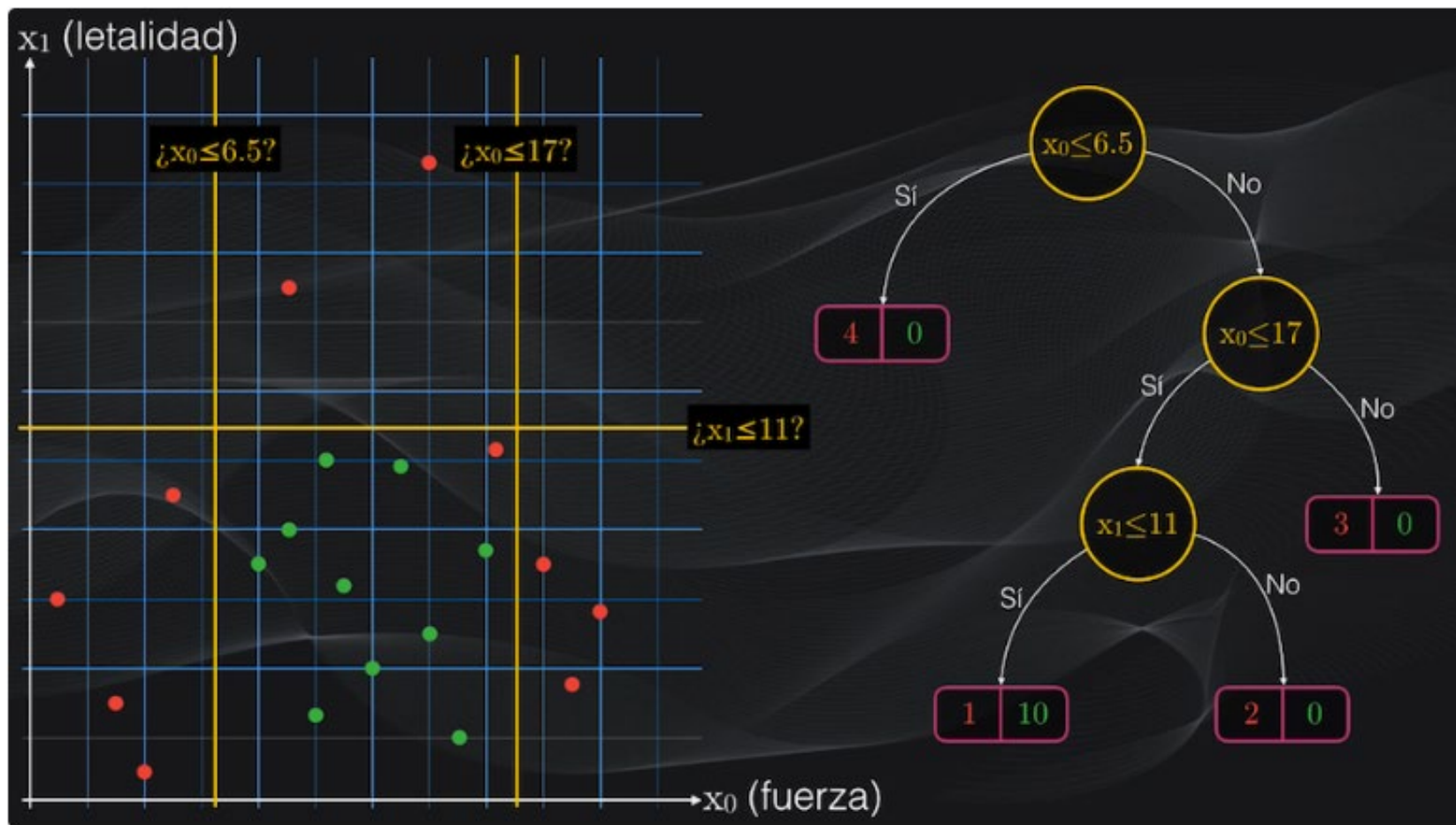
Árbol de decisión: Clasificación y CART

Luego se repite el paso anterior, una y otra vez, hasta que al final se obtengan agrupaciones lo más homogéneas posible, i.e., con puntos que pertenezcan en lo posible a una sola categoría.



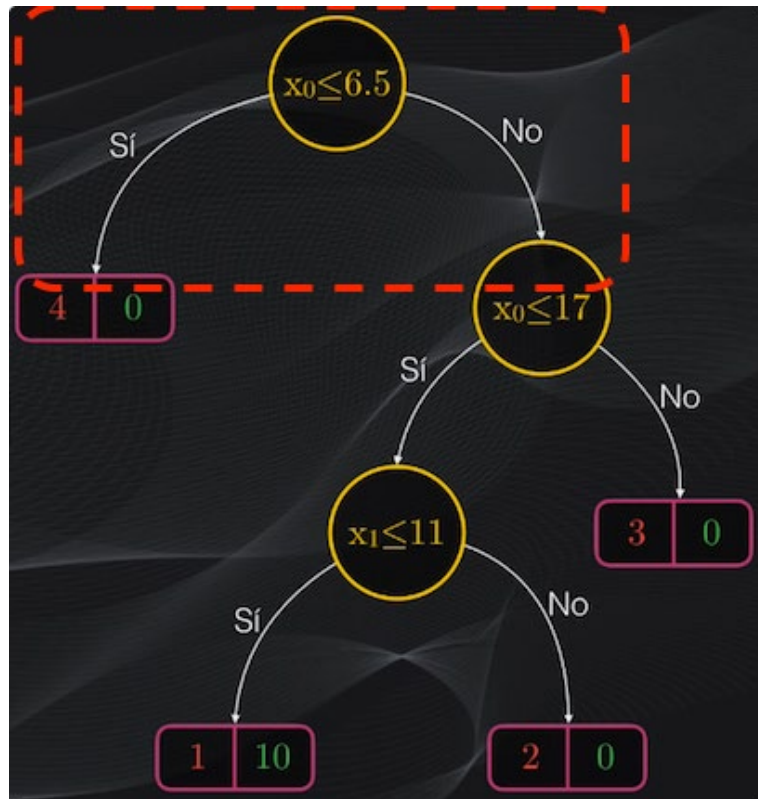
Árbol de decisión: Clasificación y CART

Esta serie de particiones la podemos representar gráficamente precisamente a través de un árbol de decisión.

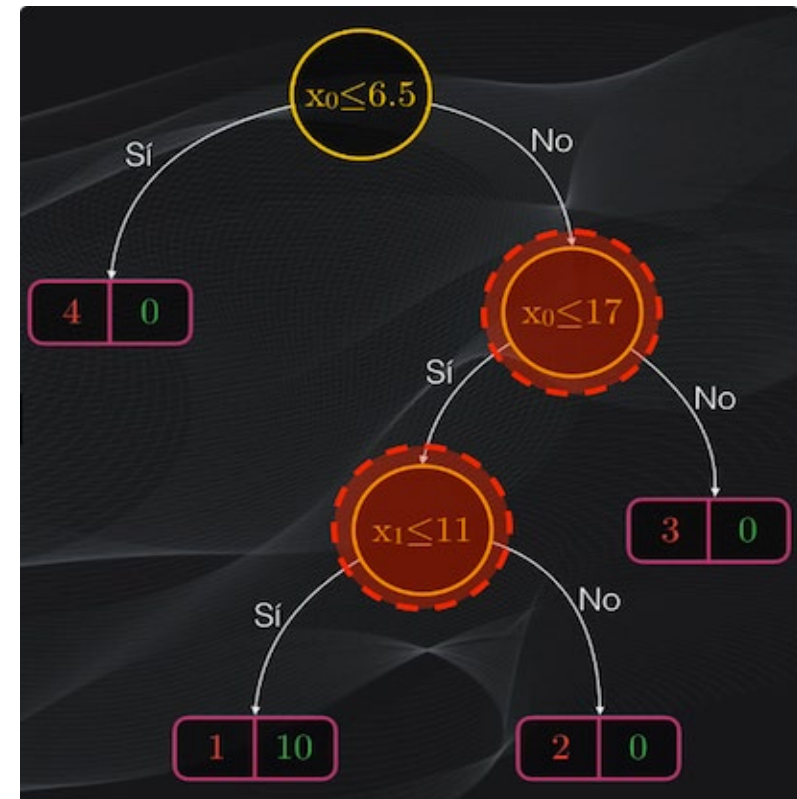


Árbol de decisión: Clasificación y CART

El punto de partida del árbol se conoce como “*raíz*” y contiene la primera condición. *Este nodo genera la primera partición binaria*, lo que se representa gráficamente como dos flechas indicando si se cumple o no la condición.

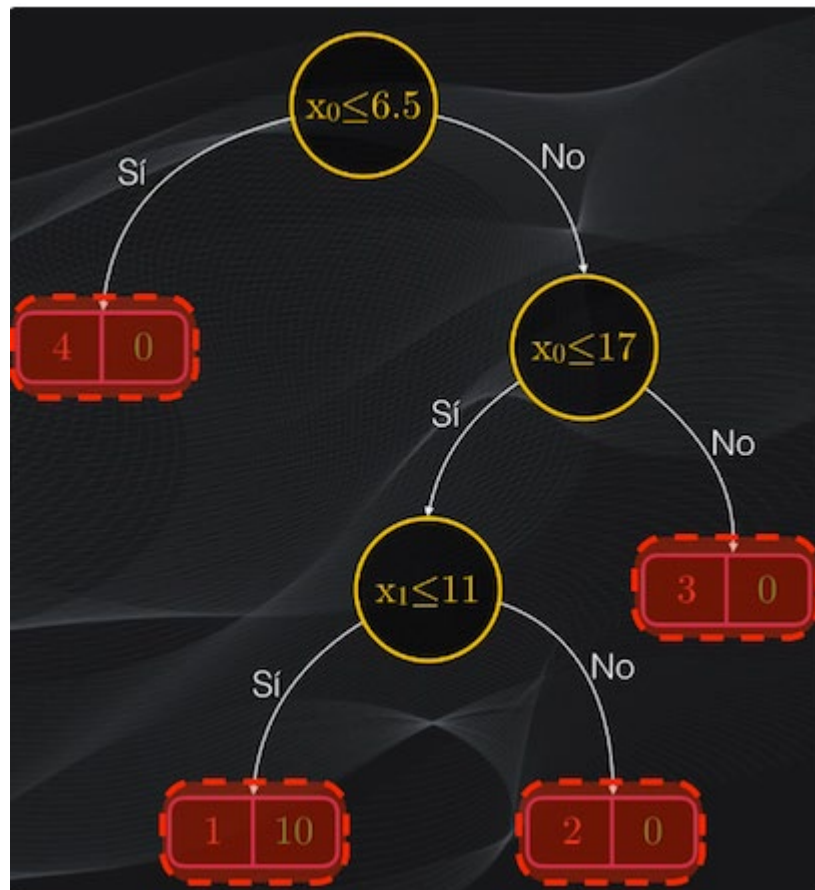


Los nodos internos o de decisión corresponden a condiciones adicionales para continuar realizando la partición.

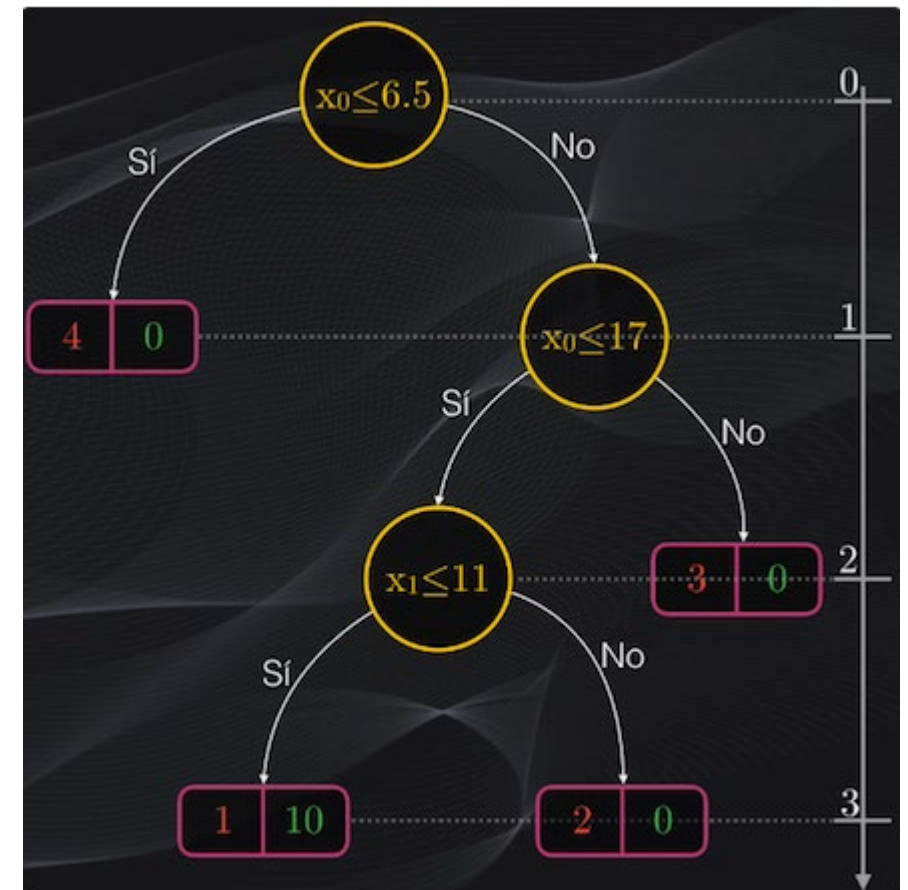


Árbol de decisión: Clasificación y CART

Los nodos hoja corresponden a las subregiones más allá de las cuales no realizaremos más particiones



La profundidad del árbol es simplemente la trayectoria más larga entre la raíz y una de las hojas.



Árbol de decisión: Clasificación y CART

Ahora nos preguntamos:

1. ¿Cómo se construye de forma automática este árbol?
2. ¿Cómo lograr que en esta construcción se generen los subgrupos más homogéneos posibles?
3. ¿Cómo se determinaron los valores para cada una de las condiciones que definen los nodos del árbol?

La respuesta es mediante el algoritmo CART, quien es capaz de generar automáticamente las particiones cada una con las agrupaciones más homogéneas posible.

Árbol de decisión: Clasificación y CART

Índice de Gini: una medida de homogeneidad

Para medir esta homogeneidad se usa el índice Gini, que mide el grado de “impureza” de un nodo.

Índices iguales a cero indican nodos puros (los datos pertenecen a una sola categoría).

Índices mayores que cero y con valores hasta de uno indican nodos con impurezas (con datos de más de una categoría).

$$\text{Índice Gini} = 1 - (\text{probabilidad categoría 1})^2 - (\text{probabilidad categoría 2})^2$$



$$1 - (10/10)^2 - (0/10)^2 = 1 - 1 - 0 = 0$$



$$1 - (0/10)^2 - (10/10)^2 = 1 - 0 - 1 = 0$$



$$1 - (7/10)^2 - (3/10)^2 = 1 - 0.49 - 0.09 = 0.42$$



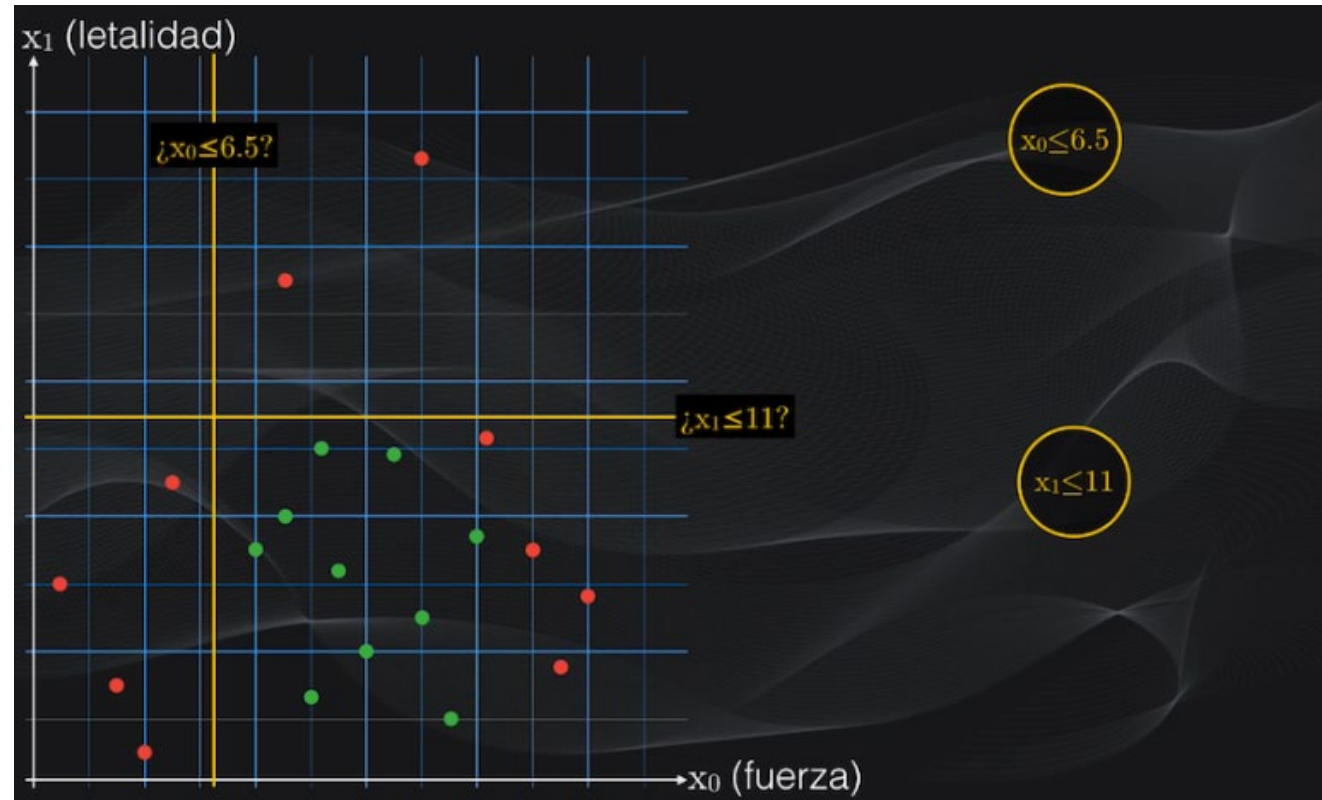
$$1 - (5/10)^2 - (5/10)^2 = 1 - 0.25 - 0.25 = 0.5$$

Árbol de decisión: Clasificación y CART

Analicemos dos posibles particiones iniciales con nuestro ejemplo de los personajes de Dragon Ball Super:

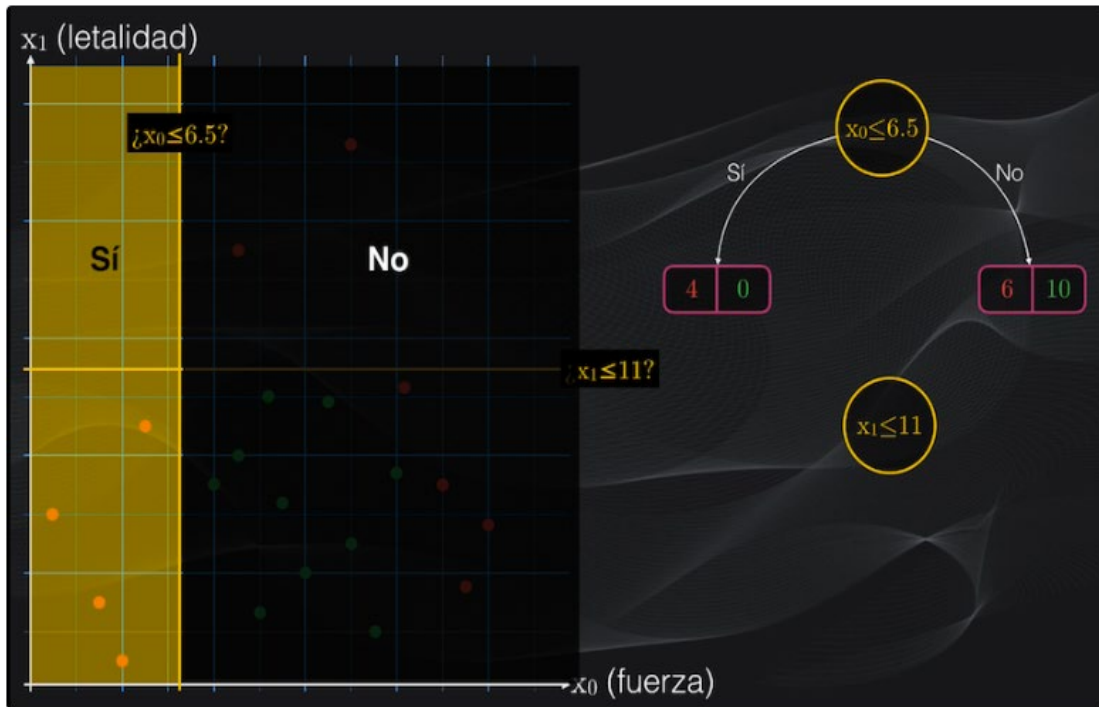
$$(1) x_0 \leq 6.5$$

$$(2) x_1 \leq 11$$

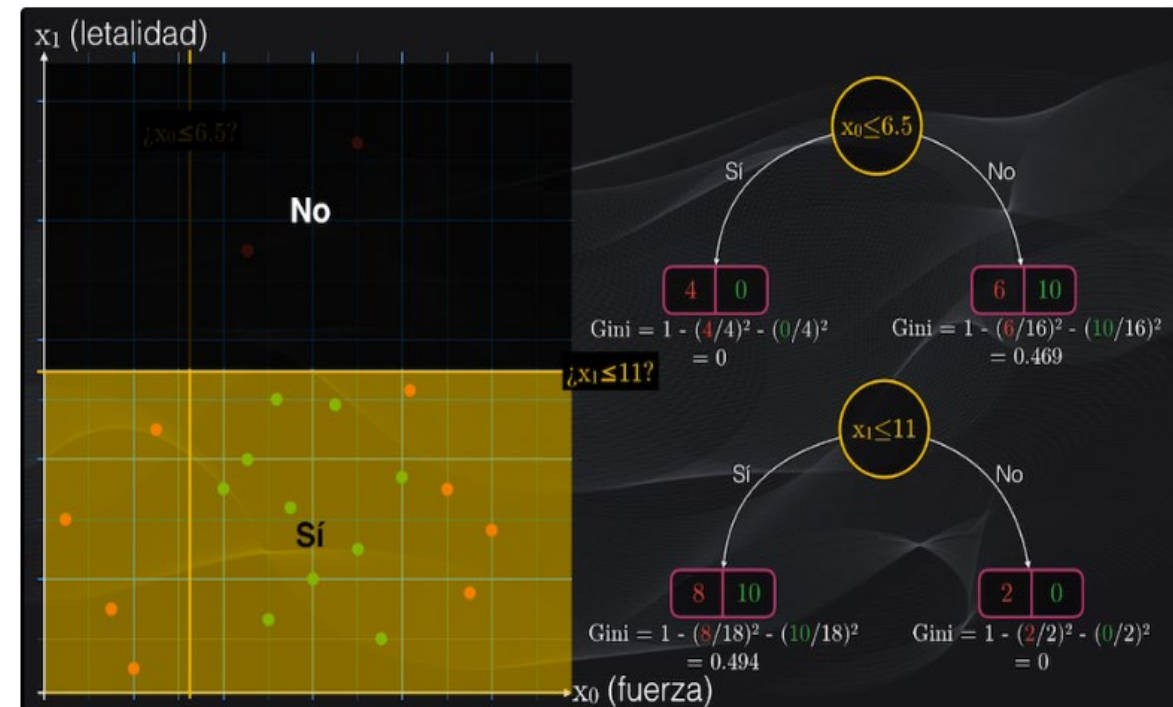


Árbol de decisión: Clasificación y CART

Para el primer umbral veremos que la partición del lado izquierdo contendrá únicamente 4 puntos rojos y 0 puntos verde, mientras que la partición del lado derecho tendrá 6 puntos rojos y 10 verdes.



Para el segundo umbral veremos que el nodo izquierdo será puro, pues contendrá 2 puntos rojos y 0 verde, mientras que el nodo derecho contendrá impurezas: 8 puntos rojos y 10 verdes.



Árbol de decisión: Clasificación y CART

Función de costo: ¿cuál es la mejor partición?

Para saber cuál de estas dos particiones es la mejor el algoritmo CART define una función de costo que asigna un puntaje al nodo padre, usando el promedio ponderado de los índices Gini individuales de sus nodos hijos.

Para calcular la función de costo en cada caso debemos obtener el valor promedio ponderado de la impureza de los nodos hijo, a izquierda y derecha.



Árbol de decisión: Clasificación y CART

Esto se calcula tomando el índice Gini correspondiente a cada nodo y multiplicándolo por el resultado de dividir el número de datos pertenecen a la nueva agrupación entre el número total de datos antes de la partición.

Así, para la primera partición, el nodo hijo del lado izquierdo tiene un índice Gini igual a cero, el número de datos resultantes de la partición es 4 puntos rojos y cero verdes, y el número total de datos antes de la partición es simplemente el conjunto de datos original (20 puntos). Así que para este nodo hijo se tiene una impureza ponderada igual a cero.

$$Impureza_{izquierda_{ponderada}} = 0 \cdot \frac{4 + 0}{20} = 0$$

Para el nodo del lado derecho el índice Gini era 0.469, tras la partición se obtuvieron 6 puntos rojos y 10 verdes, y el número total de puntos antes de la partición sigue siendo 20, lo que nos arroja una impureza ponderada de 0.375.

$$Impureza_{derecha_{ponderada}} = 0.469 \cdot \frac{6 + 10}{20} = 0.375$$

Árbol de decisión: Clasificación y CART

Al sumar estos valores ponderados de las impurezas de cada uno de estos nodos obtenemos el valor de la función de costo del nodo padre (0.375).

$$Costo_{partición_1} = 0 + 0.375 = 0.375$$

Si repetimos el mismo procedimiento para la segunda opción, calculamos las impurezas ponderadas individuales de los nodos hijos, y las sumamos, obtenemos un valor para la función de costo igual a 0.446

$$Impureza_{izquierda_ponderada} = 0.494 \cdot \frac{8 + 10}{20} = 0.446$$

$$Impureza_{derecha_ponderada} = 0 \cdot \frac{2 + 0}{20} = 0$$

$$Costo_{partición_2} = 0.446 + 0 = 0.446$$

Árbol de decisión: Clasificación y CART

El funcionamiento en detalle del algoritmo CART

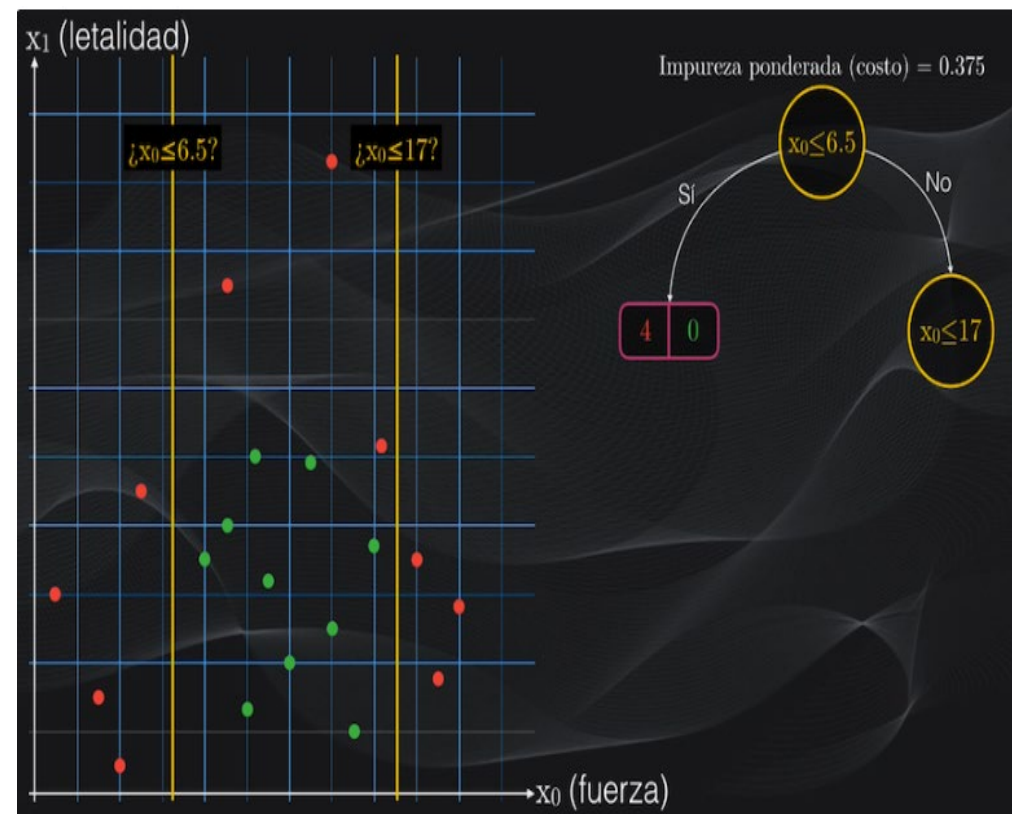
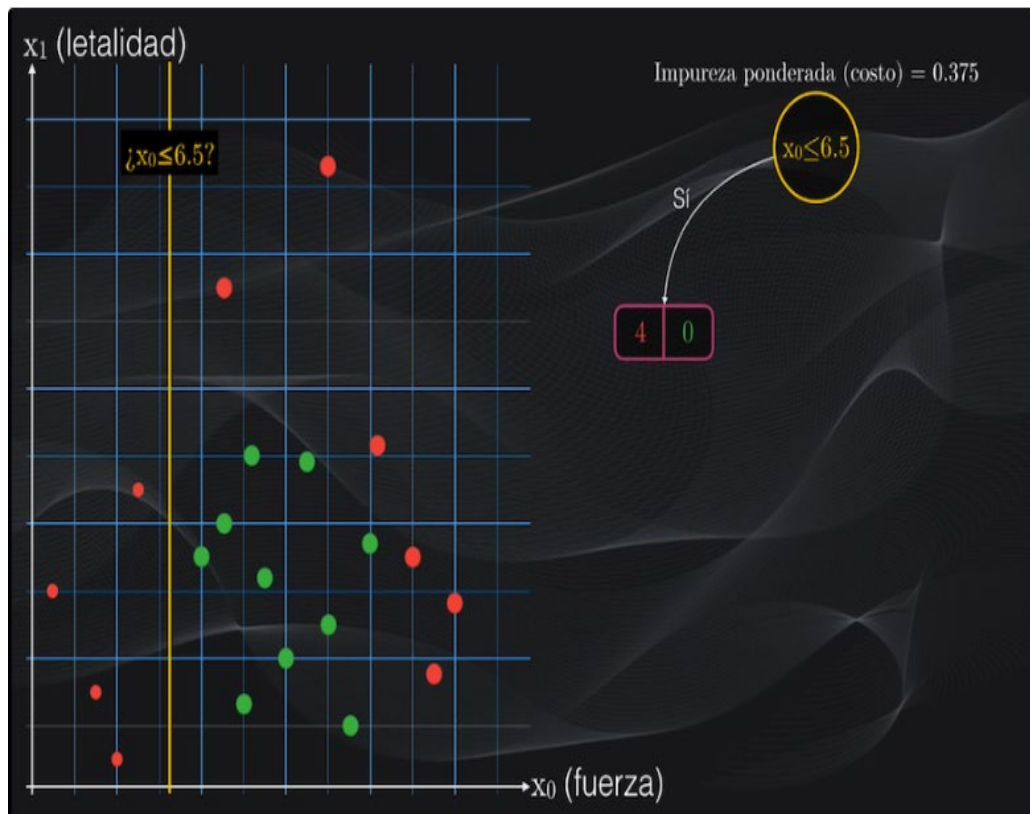
Ahora tenemos un criterio para definir cuál de las dos particiones es mejor: *¡Elegimos aquella que tenga el menor valor posible para la función de costo, lo que indica un menor nivel de impureza y por tanto una mejor clasificación!*

Base del *algoritmo CART para la clasificación con Árboles de Decisión*. Este mismo procedimiento se aplica iterativamente hasta lograr separar adecuadamente los datos, i.e., hasta generar las agrupaciones lo más homogéneas posible.

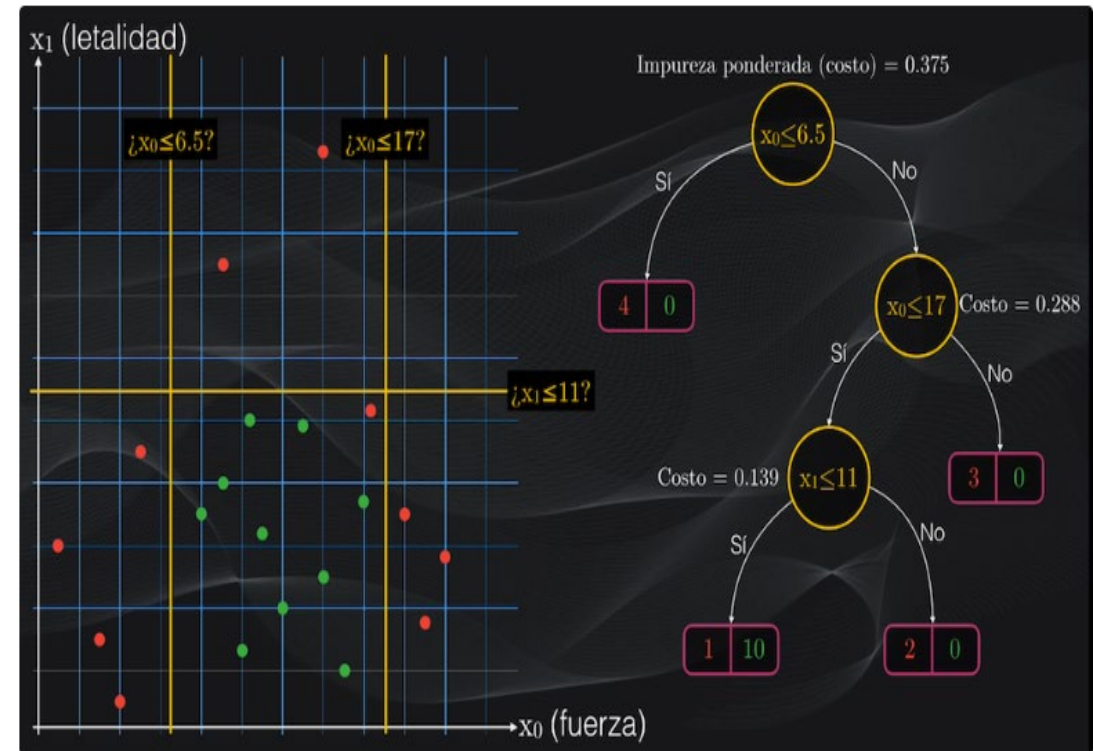
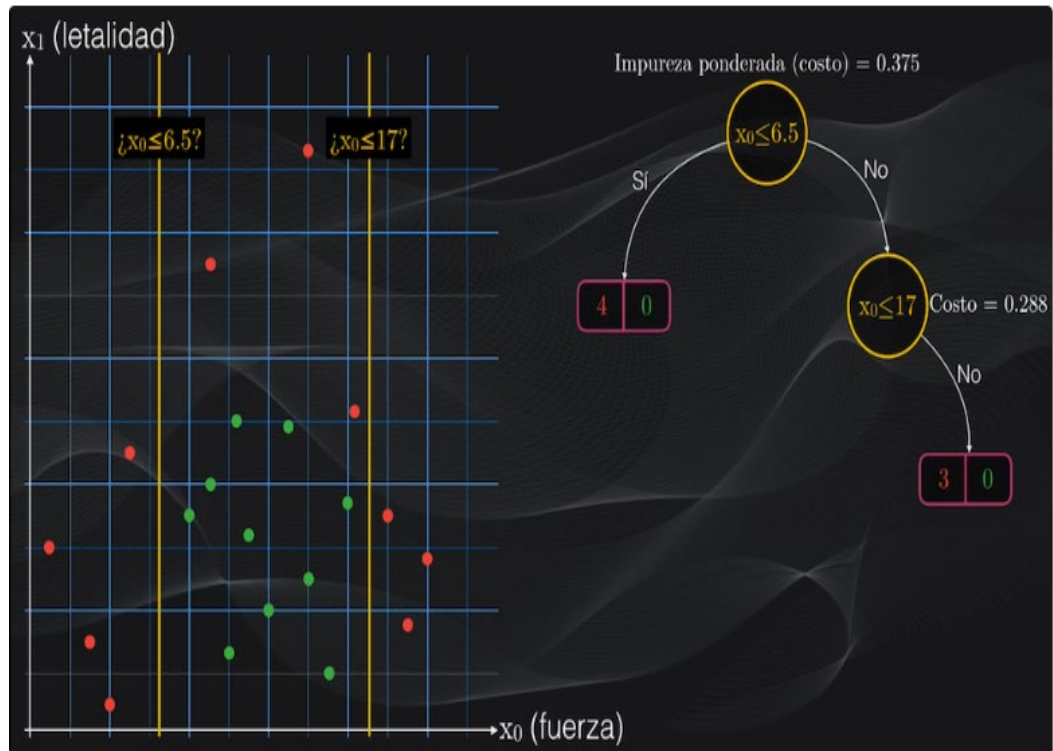
Así que si volvemos al nodo raíz seleccionado, vemos que el nodo hijo de la izquierda es una hoja, y por tanto no haremos más particiones. Sin embargo, el nodo de la derecha aún contiene impurezas y podemos intentar hacer más particiones.

Árbol de decisión: Clasificación y CART

Si repetimos el paso anterior, y analizamos todos los umbrales posibles, encontramos que la condición $x_0 \leq 17$ es la que generará el menor costo posible.

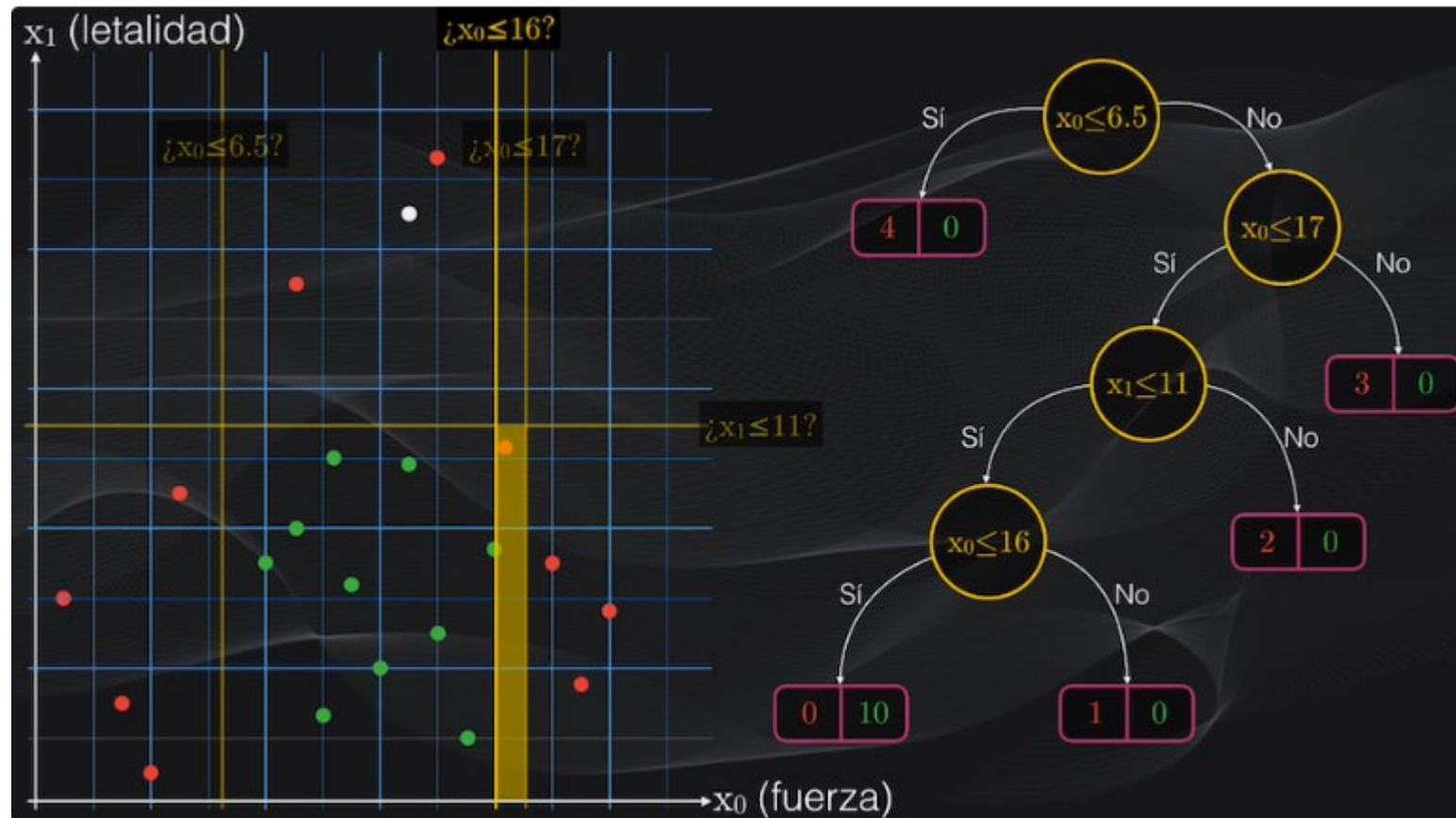


Árbol de decisión: Clasificación y CART



Árbol de decisión: Clasificación y Sobreajuste

Cuando las regiones obtenidas son muy pequeñas, es muy probable que al introducir un nuevo dato para ser analizado por el modelo, este resulte clasificado incorrectamente.



Precisión, Exhaustividad, F1-score, Exactitud en Clasificación

		Predicted		
		Negative (0)	Positive (1)	
Actual	Negative (0)	True Negative TN	False Positive FP (Type I error)	Specificity $= \frac{TN}{TN + FP}$
	Positive (1)	False Negative FN (Type II error)	True Positive TP	Recall, Sensitivity, True positive rate (TPR) $= \frac{TP}{TP + FN}$
		Accuracy $= \frac{TP + TN}{TP + TN + FP + FN}$	Precision, Positive predictive value (PPV) $= \frac{TP}{TP + FP}$	F1-score $= 2 \times \frac{Recall \times Precision}{Recall + Precision}$

Precisión, Exhaustividad, F1-score, Exactitud en Clasificación

Precisión:

Se refiere a la dispersión del conjunto de valores obtenidos a partir de mediciones repetidas de una magnitud. Cuanto menor es la dispersión mayor la precisión.

En forma práctica, es el porcentaje de casos positivos detectados.

Exhaustividad:

Es un valor que indica la capacidad de nuestro modelo para discriminar los casos positivos, de los negativos.

En forma práctica, es el porcentaje de casos positivos que fueron correctamente.

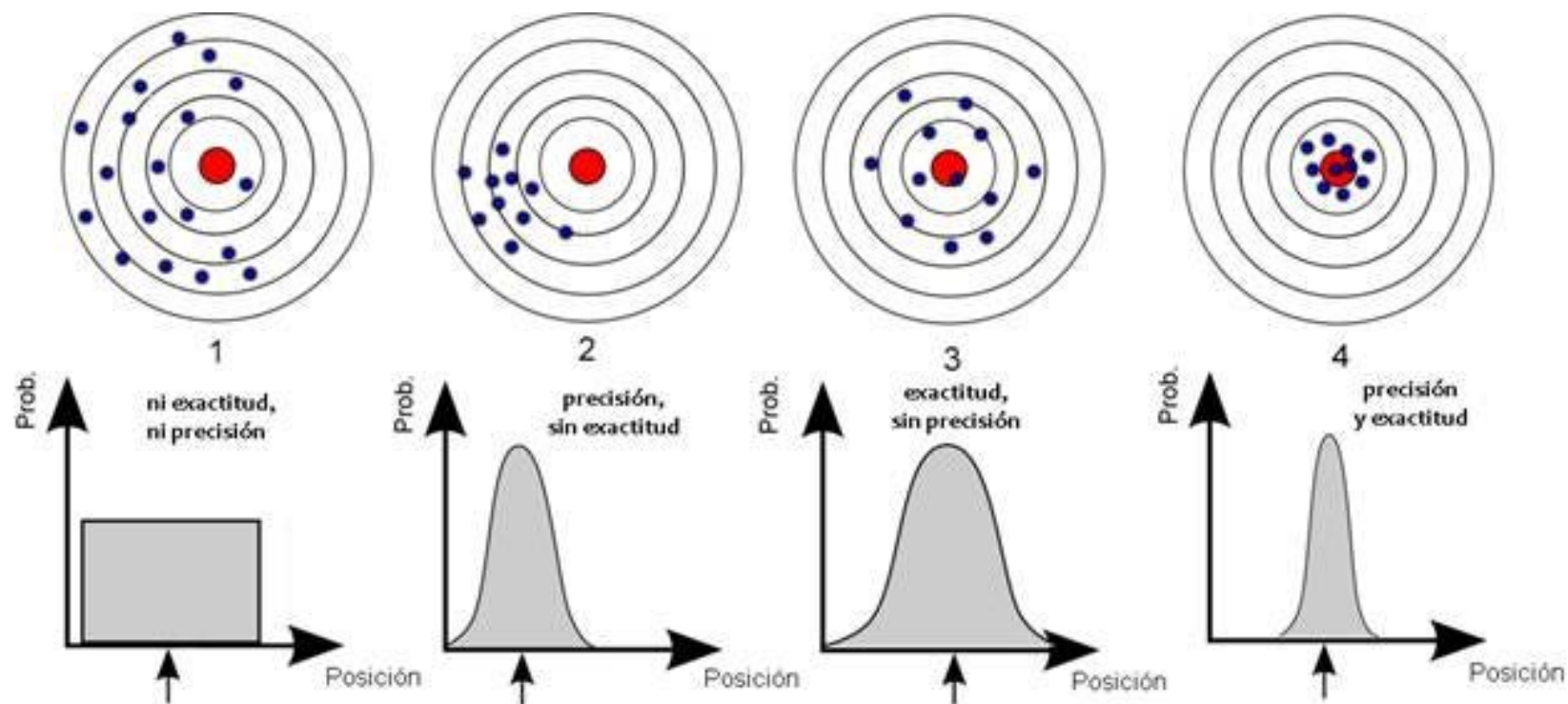
En el área de la salud decimos que la exhaustividad es la capacidad de poder detectar correctamente la enfermedad entre los enfermos.

Exactitud:

Se refiere a lo cerca que está el resultado de una medición del valor verdadero. En términos estadísticos, la exactitud está relacionada con el sesgo de una estimación.

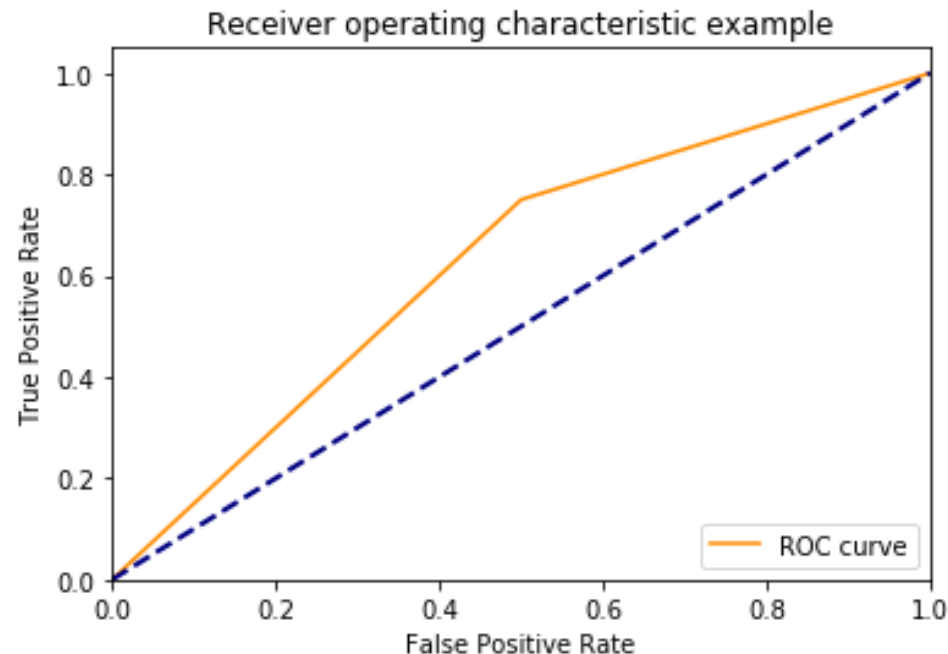
En forma práctica, es el porcentaje de predicciones positivas que fueron correctas.

Diferencia entre Exactitud y Precisión



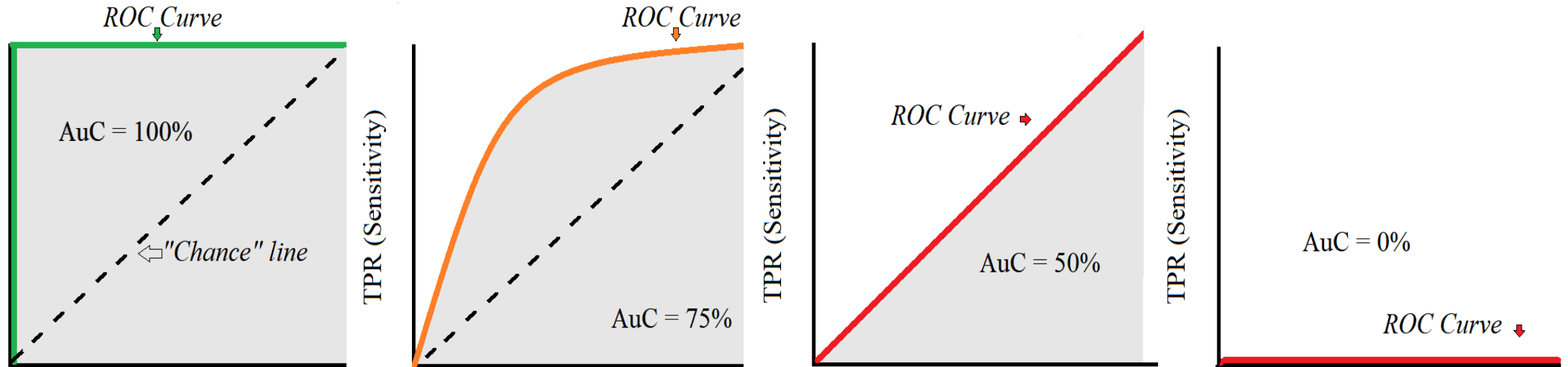
Curva ROC (Receiver Operating Characteristic)

Es un gráfico muy utilizado para evaluar modelos de aprendizaje computacional para problemas de clasificación. La gráfica representa el porcentaje de verdaderos positivos (True Positive Rate), también conocido como Recall, contra el ratio de falsos positivos (False Positive Rate). La diferencia con el resto de métricas, es que en este caso, el umbral por el que se clasifica un elemento como 0 o 1, se va modificando, para poder ir generando todos los puntos de la gráfica.



AUC (Area Under Curve)

El valor de esta métrica se encuentra en un rango entre 0 y 1, donde 0 es como si tuviéramos un modelo aleatorio, es decir, que si tiráramos los dados al aire tendríamos mejor resultado, y 1 es un resultado óptimo que indica que nuestro modelo generaliza muy bien.



Precisión y Exhaustividad en Clasificación Multi-clase

Precisión por clase:

Es la proporción de predicciones correctas para esa clase entre todas las instancias que el modelo clasificó como pertenecientes a esa clase.

$$Precision_i = \frac{TP_i}{TP_i + FP_i}$$

Exhaustividad por clase:

Es la proporción de instancias correctamente clasificadas de esa clase entre todas las instancias reales de esa clase.

$$Exhaustividad_i = \frac{TP_i}{TP_i + FN_i}$$

Precisión total:

$$Precision = \frac{1}{n} \sum_{i=1}^n Precision_i$$

Exhaustividad total:

$$Exhaustividad = \frac{1}{n} \sum_{i=1}^n Exhaustividad_i$$

Precisión y Exhaustividad en Clasificación Multi-clase

Supongamos que tenemos un clasificador de frutas que puede identificar manzanas, bananas y naranjas. Tenemos 100 frutas en total y el modelo hace las siguientes predicciones:

Matriz de confusión:		Predicción		
		Manzana	Banana	Naranja
Real	Manzana	28	2	5
	Banana	3	30	2
	Naranja	4	1	25

$$\text{Precisión (Manzana)} = \frac{28}{28+3+4} = \frac{28}{35} \approx 0,80$$

$$\text{Exhaustividad (Manzana)} = \frac{28}{28+2+5} = \frac{28}{35} \approx 0,80$$

$$\text{Precisión (Banana)} = \frac{30}{2+30+1} = \frac{30}{33} \approx 0,91$$

$$\text{Exhaustividad (Banana)} = \frac{30}{3+30+2} = \frac{30}{35} \approx 0,86$$

$$\text{Precisión (Naranja)} = \frac{25}{5+2+25} = \frac{25}{32} \approx 0,78$$

$$\text{Exhaustividad (Naranja)} = \frac{25}{4+1+25} = \frac{25}{30} \approx 0,83$$

Validación Cruzada (Cross-Validation)

Proceso:

1. Se divide el conjunto de datos en **k particiones (folds)**.
2. Se entrena el modelo en **k-1 particiones** y se evalúa en la partición restante.
3. El proceso se repite **k veces**, cambiando la partición usada para evaluación.
4. El rendimiento final es el **promedio** de los resultados de las k evaluaciones.

Uso

▪ Optimización de hiperparámetros:

Se prueba cada combinación de parámetros (ej. profundidad del árbol, C y kernel en SVM, k en KNN) usando CV, y se selecciona la que dé mejor rendimiento medio.

▪ Evaluación final del modelo:

CV entrega una estimación robusta del desempeño en datos no vistos, reduciendo el sesgo de una sola división entrenamiento/test.

Validación Cruzada (Cross-Validation)

Configuración Inicial

Dataset Completo:



Entrenamiento (80%):



Test (20%):



Iteración 2 de 5 (5-Fold CV)

Dataset Completo:



Entrenamiento (80%):



Test (20%):



Iteración 3 de 5 (5-Fold CV)

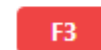
Dataset Completo:



Entrenamiento (80%):

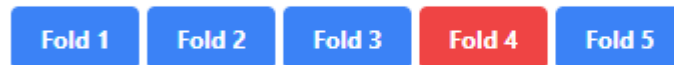


Test (20%):



Iteración 4 de 5 (5-Fold CV)

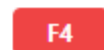
Dataset Completo:



Entrenamiento (80%):



Test (20%):



Iteración 5 de 5 (5-Fold CV)

Dataset Completo:



Entrenamiento (80%):



Test (20%):



Validación Monte Carlo

Proceso:

1. Se generan **m particiones aleatorias** en conjuntos de entrenamiento y validación.
2. Se entrena y evalúa el modelo en cada partición
3. Se promedian los resultados de las métricas utilizadas y varianza.

Uso

- **Evaluación final del modelo:**
Se busca medir la **variabilidad del rendimiento** (al repetir muchas divisiones aleatorias)
- Se tiene un **conjunto de datos muy grande**, y dividir en k folds puede ser costoso.

Validación cruzada y Monte Carlo cumplen dos roles: primero nos ayudan a **optimizar el modelo** (elegir hiperparámetros), y después nos permiten **estimar la capacidad de generalización** antes de aplicarlo en el test final.

Consideraciones Importantes

Validación cruzada de K-Fold

- Cada observación se usa exactamente una vez para test
- Mejor aprovechamiento de datos limitados
- Estimación más estable del rendimiento
- K típicamente 5 o 10

Validación Monte Carlo

- Particiones aleatorias repetidas
- Mayor flexibilidad en tamaños de conjuntos
- Puede usar observaciones múltiples veces
- M repeticiones (ej. 100 o 1000)

Pipeline: Validación interna + Evaluación del rendimiento

1. Esquema de partición de los datos

Definir el método de evaluación (ej. k-fold cross-validation o validación Monte Carlo).

- En cada iteración, se dividen los datos en un **conjunto de entrenamiento** y un **conjunto de test**.
- Esto se repite k veces (en CV) o m veces (en Monte Carlo), generando múltiples evaluaciones.

2. Optimización de hiperparámetros (validación interna)

Dentro del conjunto de entrenamiento de cada iteración, separar una **pequeña porción para validación interna**.

- Ajustar los hiperparámetros (ej. profundidad máxima en árboles, k en K-NN, C y γ en SVM).
- Elegir la configuración que logre el mejor desempeño en la validación interna.

3. Entrenamiento con parámetros óptimos

Con los hiperparámetros seleccionados, entrenar el modelo usando todo el conjunto de entrenamiento correspondiente a la iteración.

4. Evaluación global

- Evaluar el modelo en el conjunto de test de la iteración y repetir este proceso en todas las iteraciones.
- El rendimiento final se obtiene promediando las métricas de todas las particiones (k veces o m repeticiones).

Pipeline: Validación interna + Evaluación del rendimiento

Pipeline de Evaluación



1. Esquema de Partición



2. División Train/Test



3. Validación Interna



4. Optimización



5. Entrenamiento Final



6. Evaluación



7. Repetición

1. Esquema de Partición

Definir método de evaluación (k-fold CV o Monte Carlo)



1. Esquema de Partición



2. División Train/Test



3. Validación Interna



4. Optimización



5. Entrenamiento Final



6. Evaluación

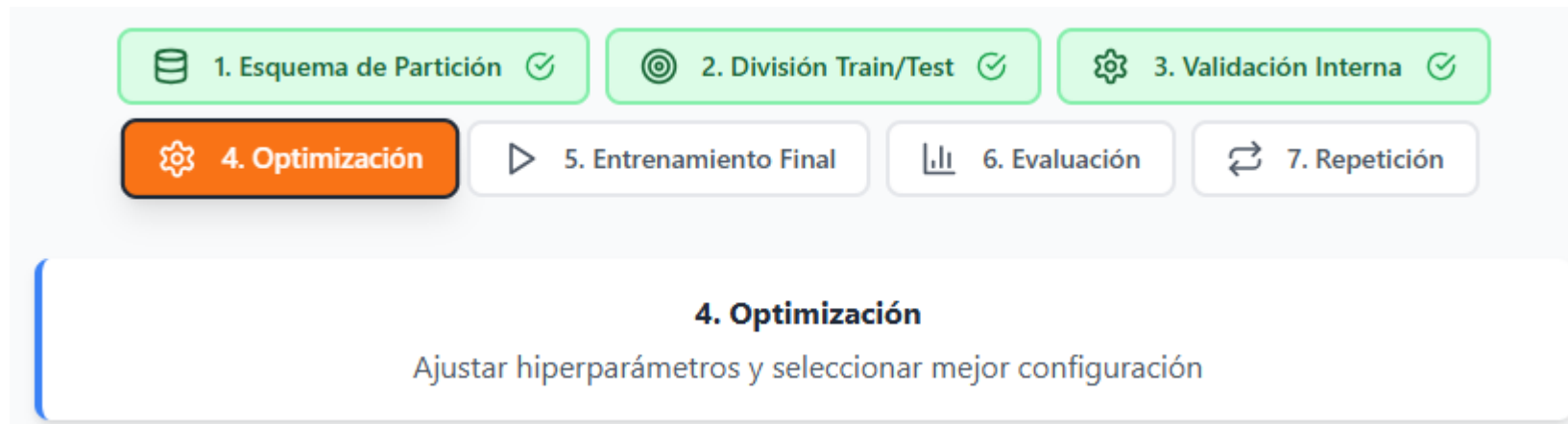
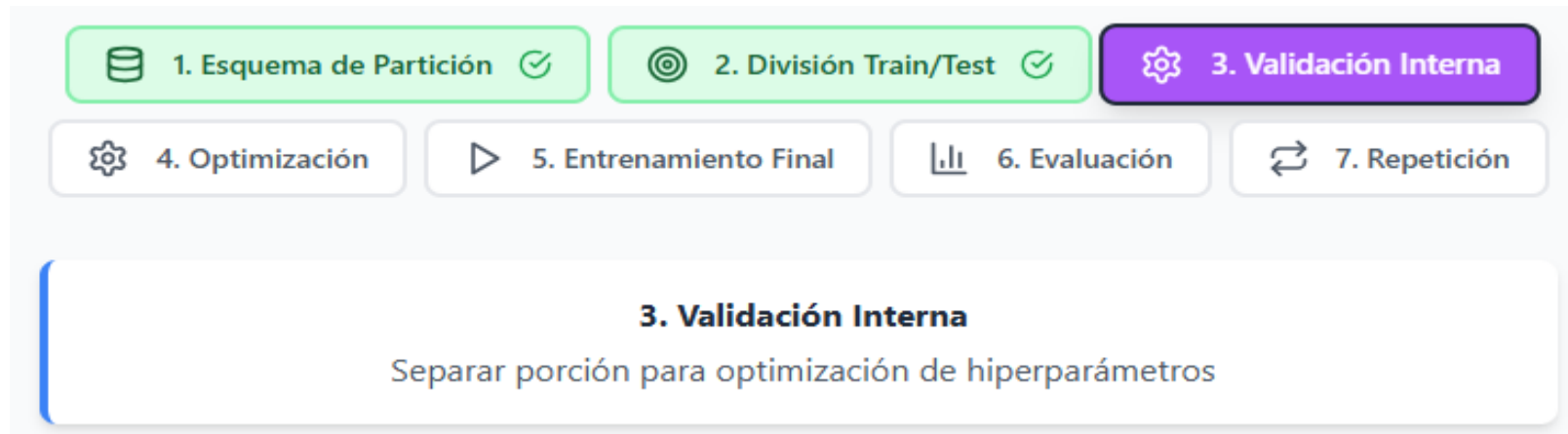


7. Repetición

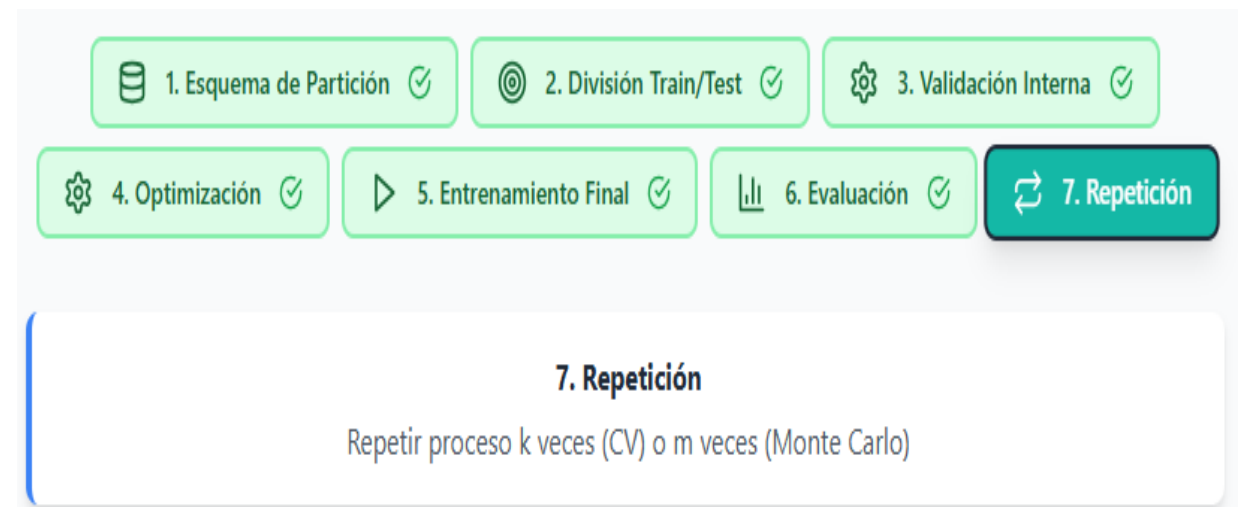
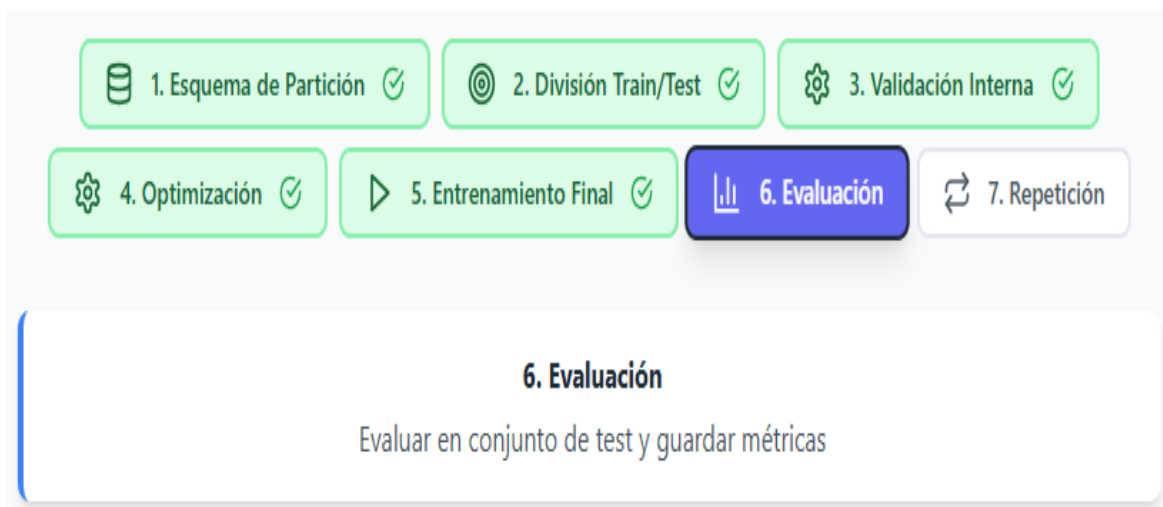
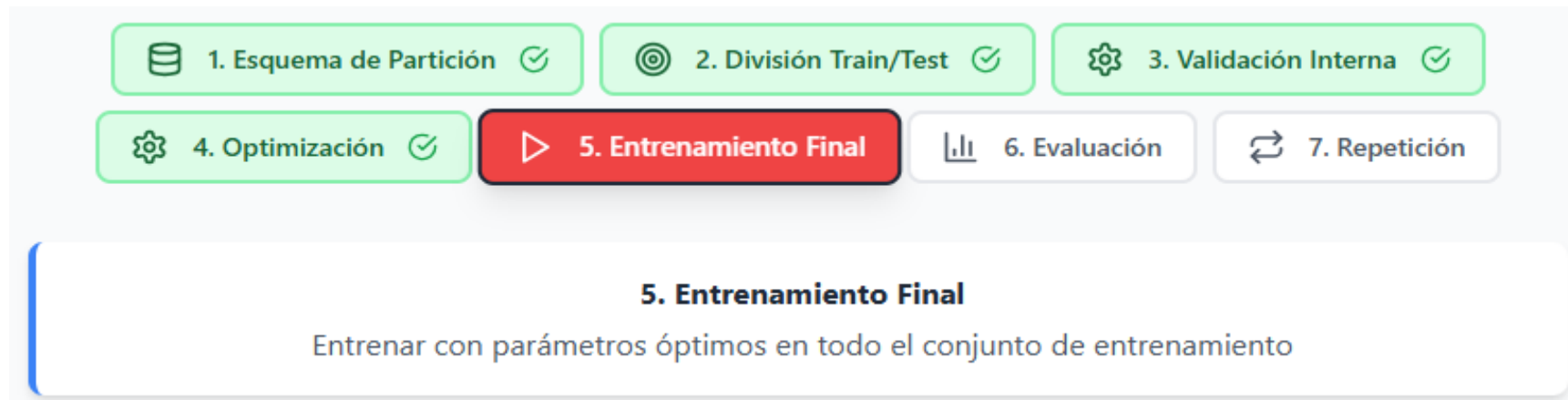
2. División Train/Test

Separar datos en entrenamiento y prueba (iteración actual)

Pipeline: Validación interna + Evaluación del rendimiento



Pipeline: Validación interna + Evaluación del rendimiento



Ejemplo en python

Conjunto de datos: *Clasificación de variedades de trigo*

Información del conjunto de datos:

El conjunto de datos comprendía granos de trigo pertenecientes a tres variedades diferentes de trigo: Kama, Rosa y Canadian, 70 elementos cada una.

Todos estos parámetros fueron de valor real continuo.

Información de atributos:

Para construir los datos, se midieron siete parámetros geométricos de los granos de trigo:

área A, perímetro P, compacidad $C = 4\pi A/P^2$, longitud del grano, ancho del grano, coeficiente de asimetría y longitud del surco del grano.

Ejemplo en python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, export_graphviz
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, ConfusionMatrixDisplay
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from graphviz import Source

target_names = {
    1:'Kama',
    2:'Rosa',
    3:'Canadian'
}
```

Ejemplo en python

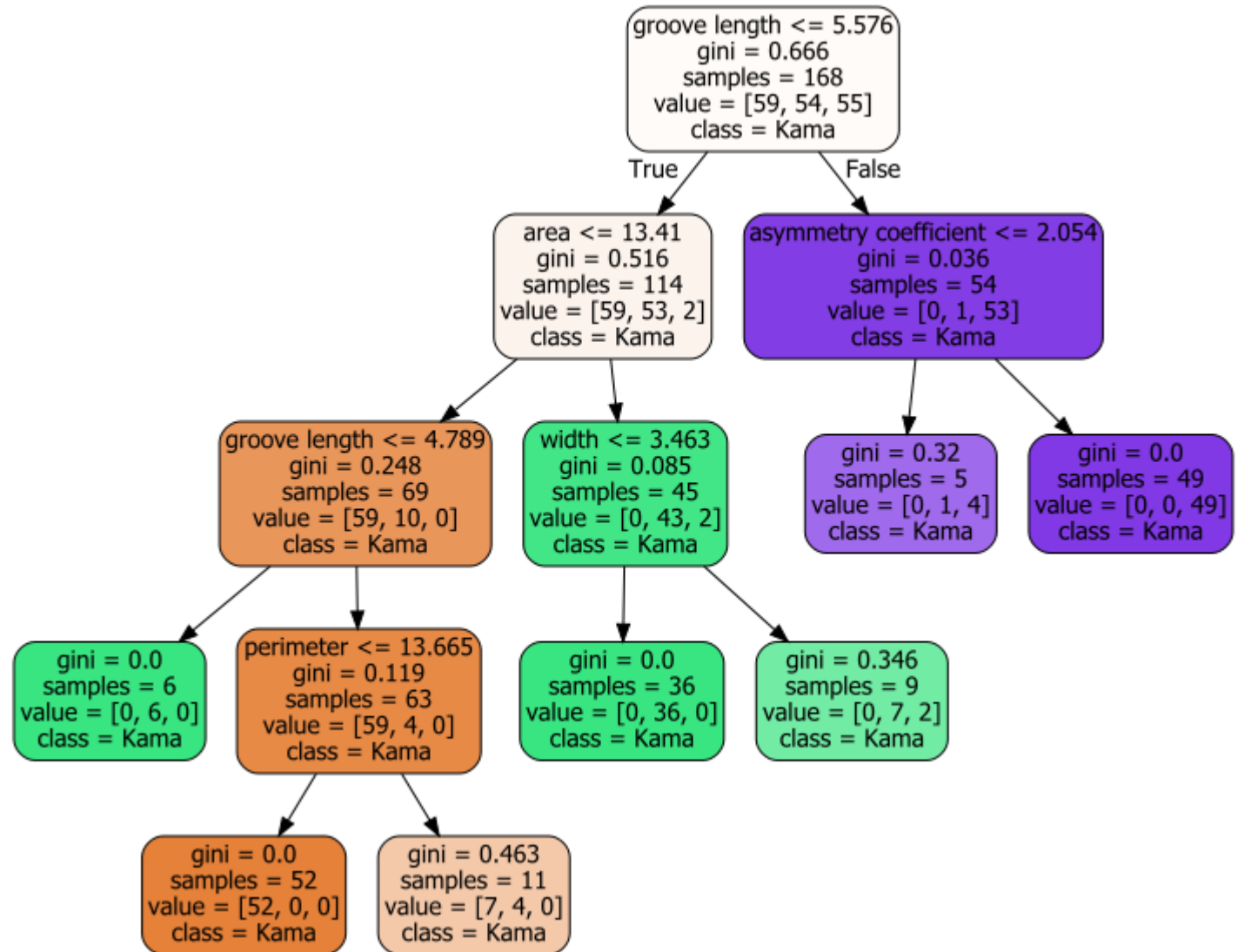
```
dataWheat = pd.read_csv("wheat.csv")  
dataWheat['category'] =  
dataWheat['category'].map(target_names)
```

```
xWheat = np.array(dataWheat[["area",  
"length", "asymmetry coefficient"]])  
yWheat = np.array(dataWheat['category'])
```

```
tree_clf = DecisionTreeClassifier(  
max_depth=4, min_samples_leaf=5,  
random_state=42)  
tree_clf.fit(xWheatTrain, yWheatTrain)
```

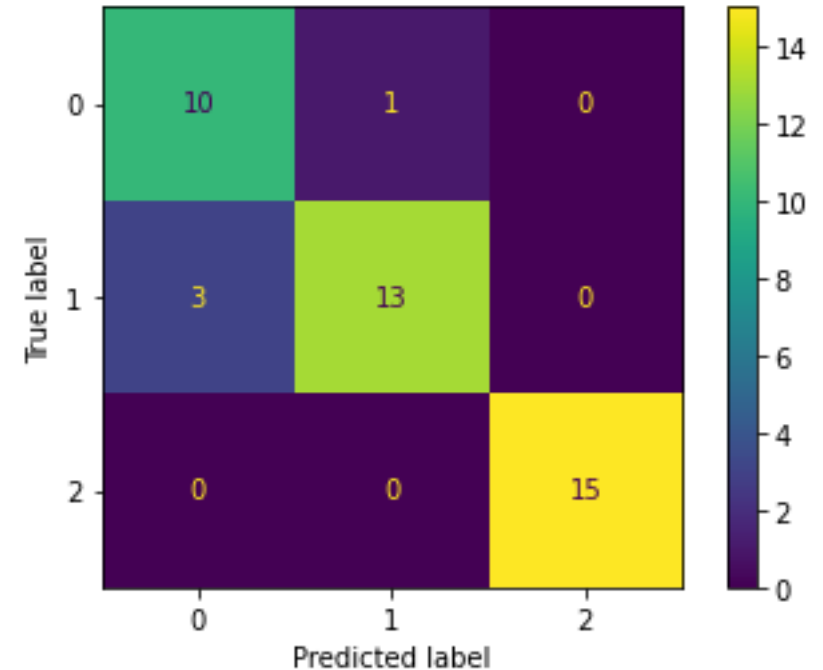
```
export_graphviz(tree_clf, out_file="wheat.dot",  
feature_names=xWheat.columns,  
class_names=yWheat, rounded=True,  
filled=True)
```

```
Source.from_file("wheat.dot")
```



Ejemplo en python

```
yWheatPred = tree_clf.predict(xWheatTest)
cm = confusion_matrix(yWheatReal, yWheatPred)
ConfusionMatrixDisplay(confusion_matrix=cm).plot()
```



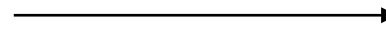
`precision_score(yWheatReal, yWheatPred, average='macro')` → 0.899

`recall_score(yWheatReal, yWheatPred, average='macro')` → 0.907

`accuracy_score(yWheatReal, yWheatPred)` → 0.904

Ejemplo en python

```
tree_clf.predict_proba(xWheatTest[0:4]).round(3)
```



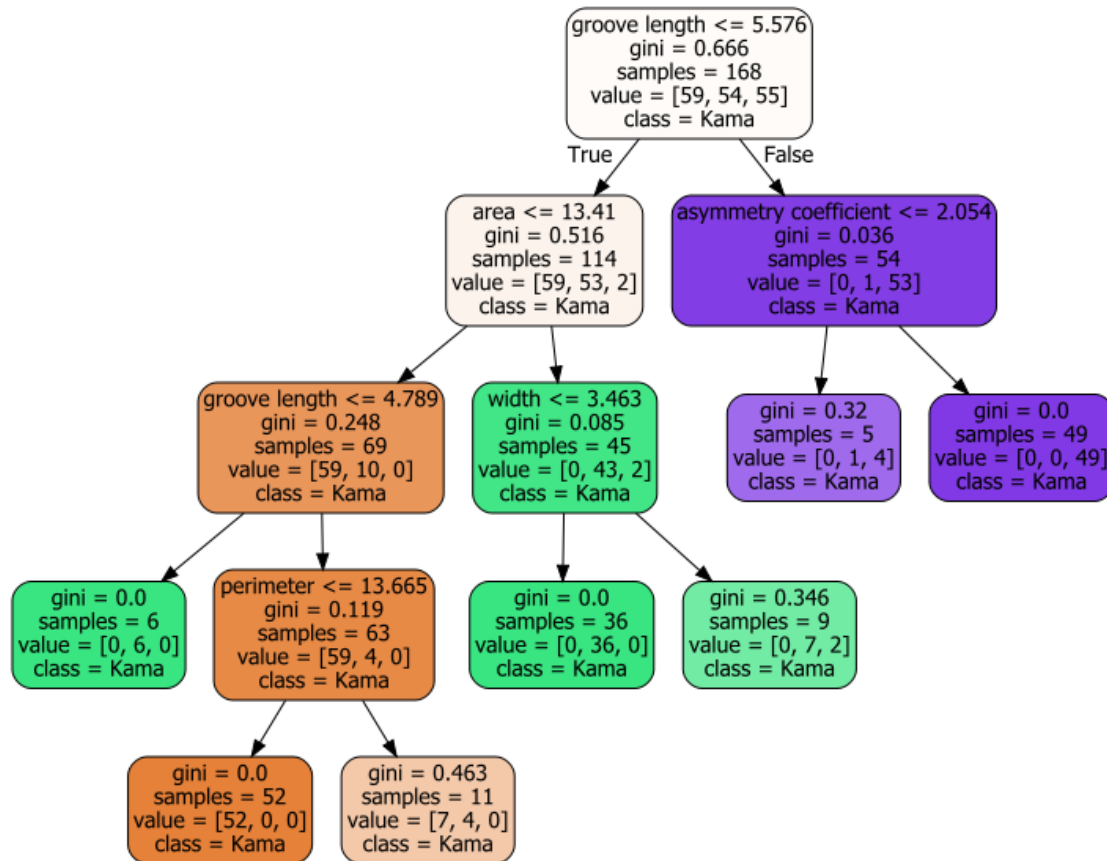
```
array([[0. , 0. , 1. ],  
       [0. , 0.2, 0.8],  
       [1. , 0. , 0. ],  
       [1. , 0. , 0. ]])
```

```
pca_pipeline = make_pipeline(StandardScaler(), PCA())  
xWheatRotated = pca_pipeline.fit_transform(xWheatTrain)  
tree_clf_pca = DecisionTreeClassifier(max_depth=4,  
min_samples_leaf=5, random_state=42)  
tree_clf_pca.fit(xWheatTrain, yWheatTrain)
```

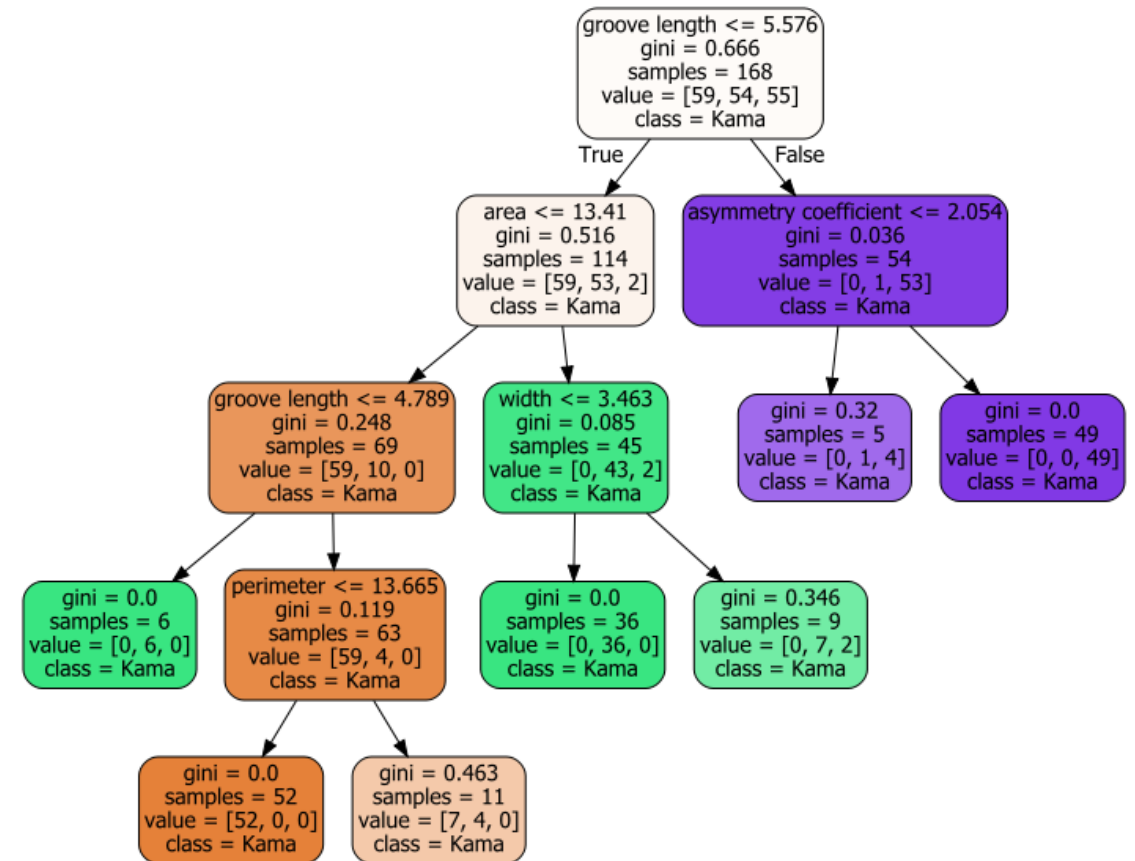
```
export_graphviz(tree_clf_pca, out_file="wheat.dot",  
feature_names=xWheat.columns,  
class_names=yWheat, rounded=True, filled=True)  
Source.from_file("wheat.dot")
```

Ejemplo en python

Sin PCA

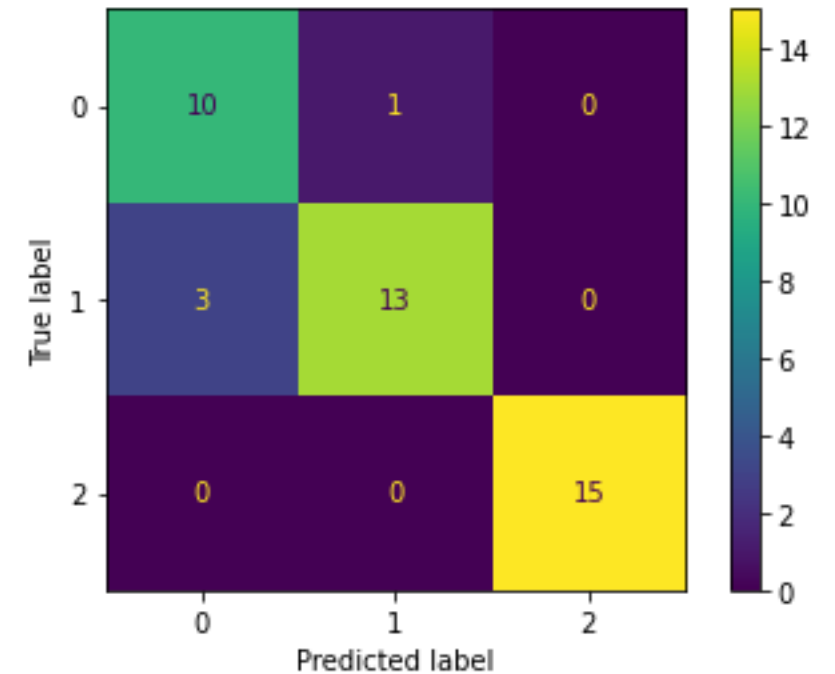


Con PCA



Ejemplo en python

```
yWheatPred = tree_clf_pca.predict(xWheatTest)
cm = confusion_matrix(yWheatReal, yWheatPred)
ConfusionMatrixDisplay(confusion_matrix=cm).plot()
```



`precision_score(yWheatReal, yWheatPred, average='macro')` → 0.899

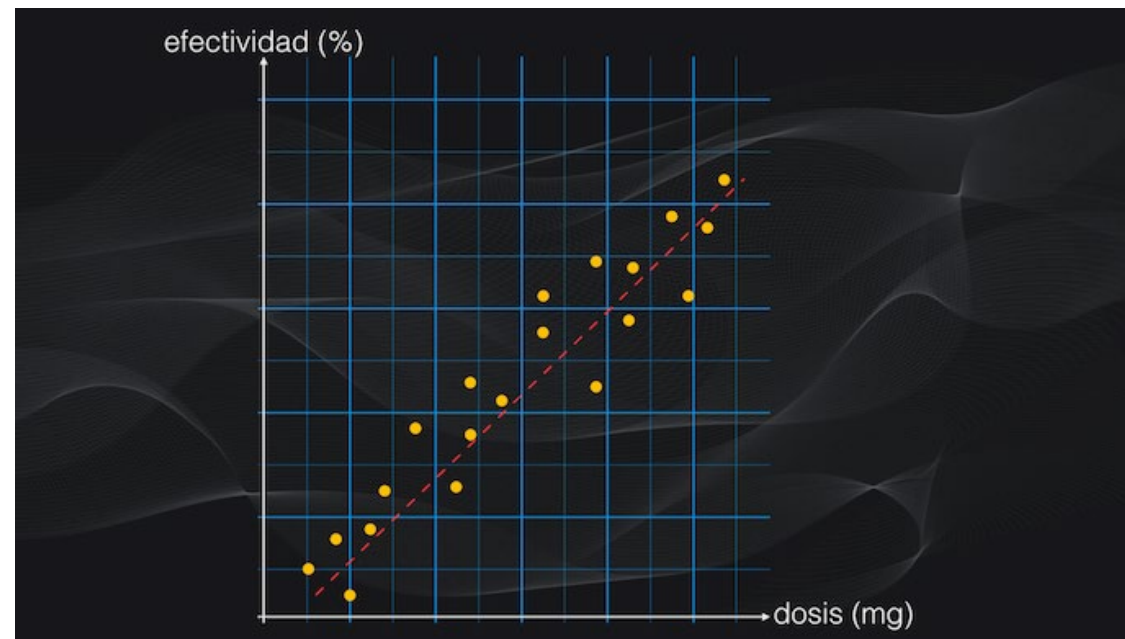
`recall_score(yWheatReal, yWheatPred, average='macro')` → 0.907

`accuracy_score(yWheatReal, yWheatPred)` → 0.904

Árbol de decisión: Regresión

Analizamos el problema de los medicamentos, considerando una única característica (entrada) que es la dosis en miligramos y queremos predecir el porcentaje de efectividad (salida).

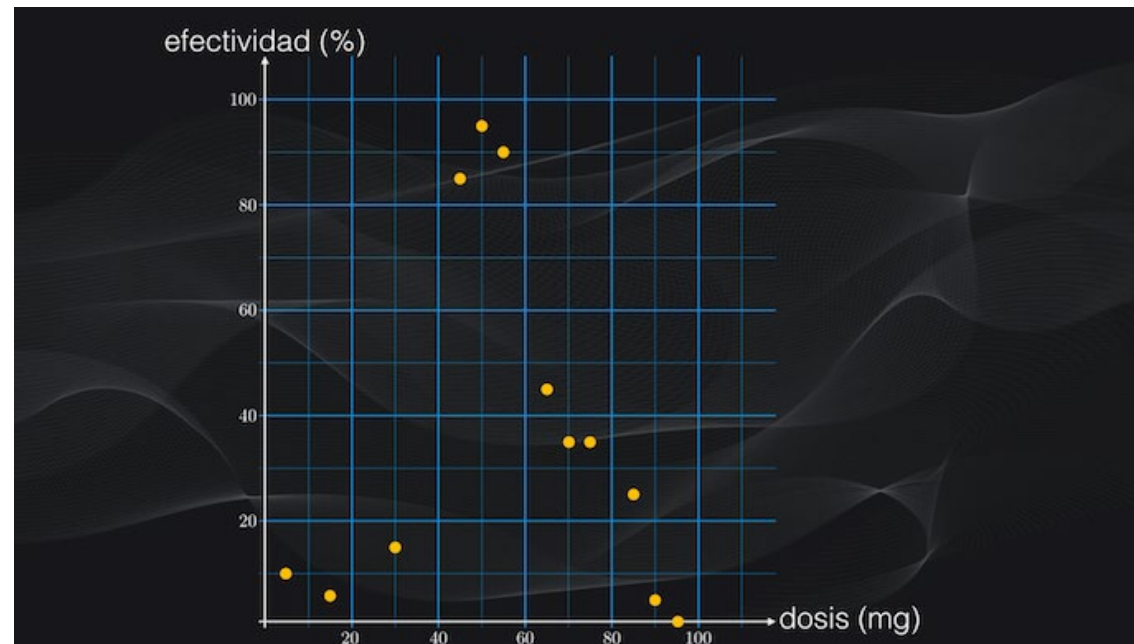
Si la relación entre estas dos variables (características) es lineal, esto indica que a mayor dosis mayor efectividad, el problema se puede resolver fácilmente usando el algoritmo de regresión lineal.



Árbol de decisión: Regresión

Ahora lo modificamos para llevarlo a las condiciones reales, supongamos que la relación entre la efectividad y la dosis tiene el comportamiento no lineal.

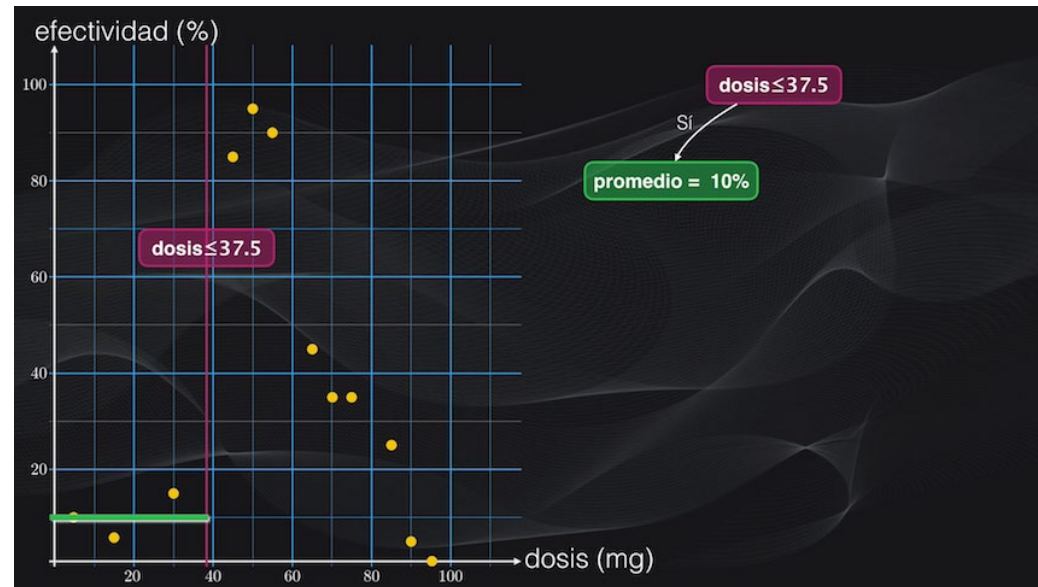
Para un rango de dosis la efectividad es baja, para otro rango tiene un valor alto y para otro tiene un valor medio.



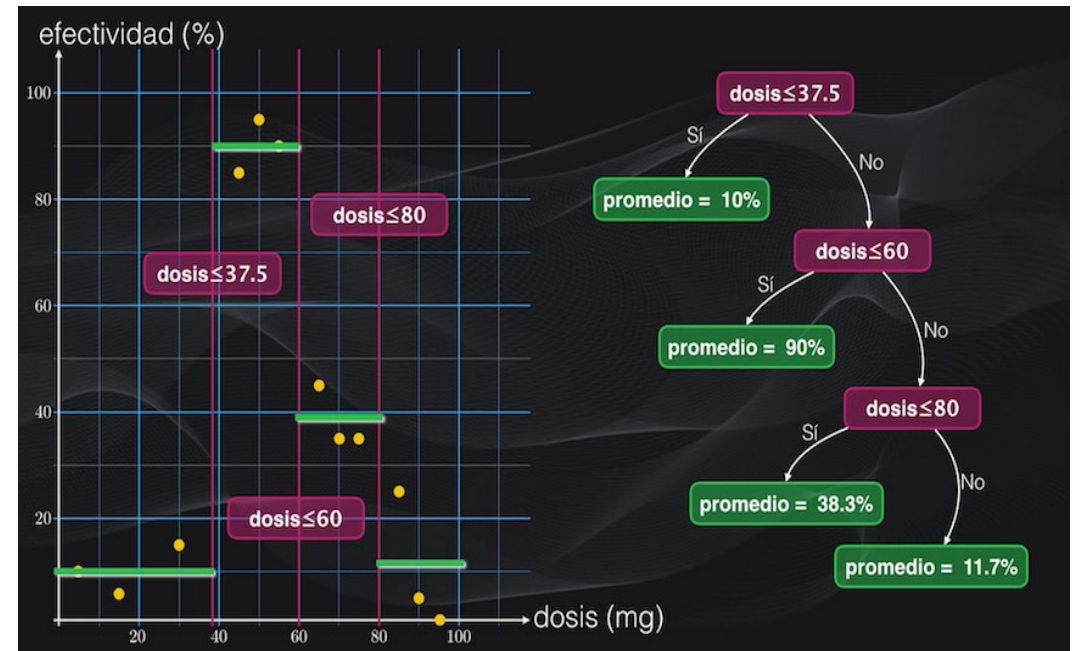
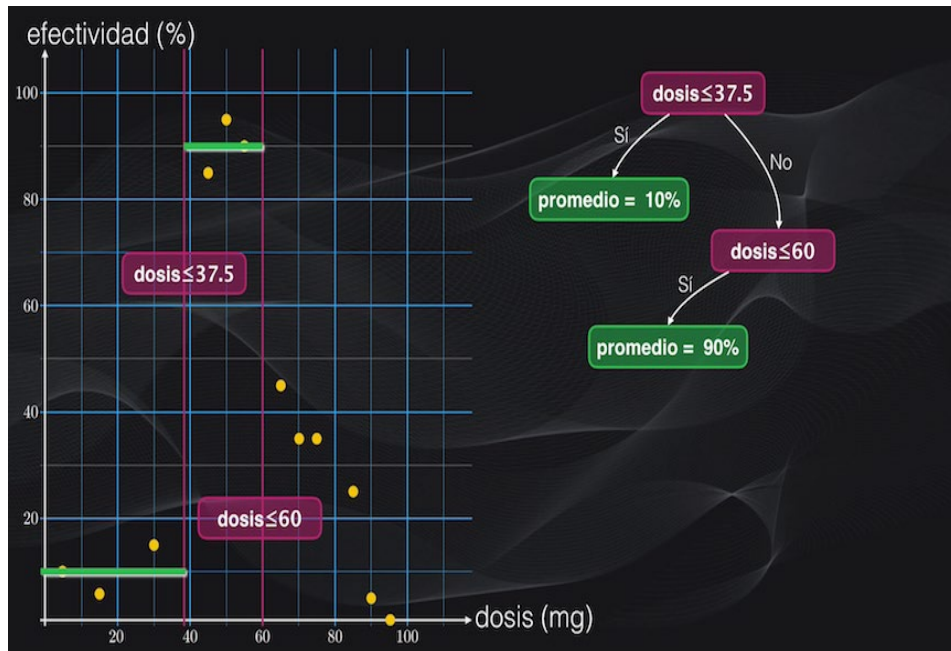
Árbol de decisión: Regresión y CART

En la regresión con árboles de decisión de manera iterativa se van realizando particiones binarias sobre el espacio de características buscando que en cada región obtenida cada vez se tengan distribuciones lo más homogéneas posible, i.e., datos con niveles de efectividad del medicamento lo más cercanos entre sí.

Si establecemos por ejemplo una condición de una dosis menor o igual a 37.5 mg, los puntos a la izquierda no estarán muy dispersos y en promedio tendrán una efectividad del 10%, pero los del lado derecho, que no cumplen la condición, estarán más dispersos, y por tanto el promedio de la efectividad no será una representación muy precisa de esta región.

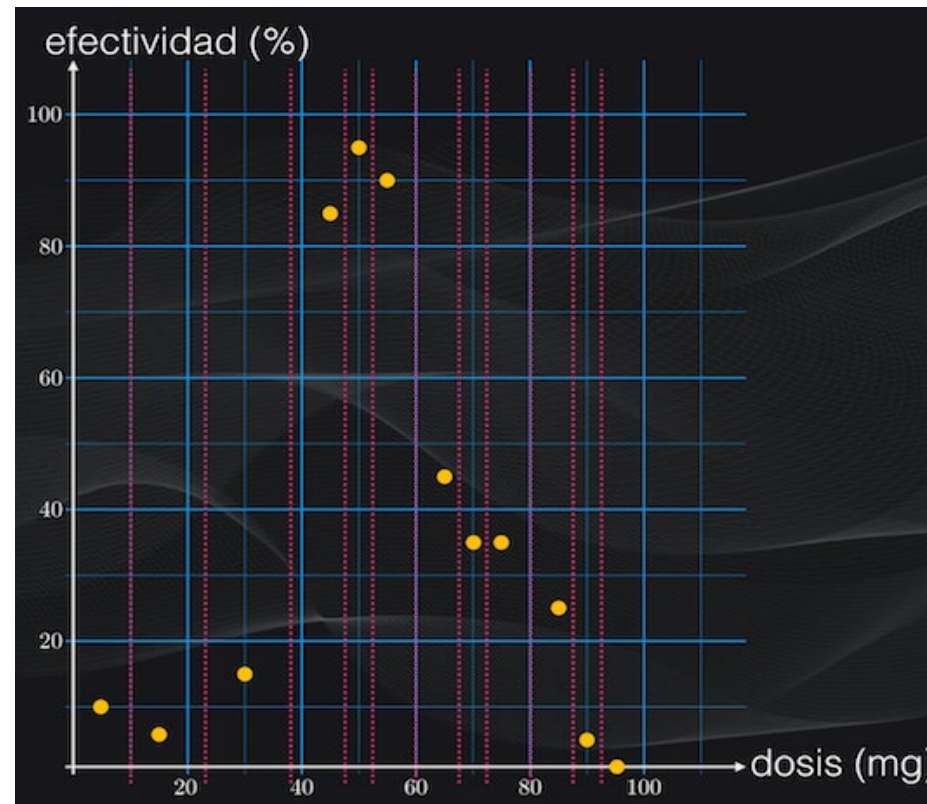


Árbol de decisión: Regresión y CART



Árbol de decisión: Regresión y CART

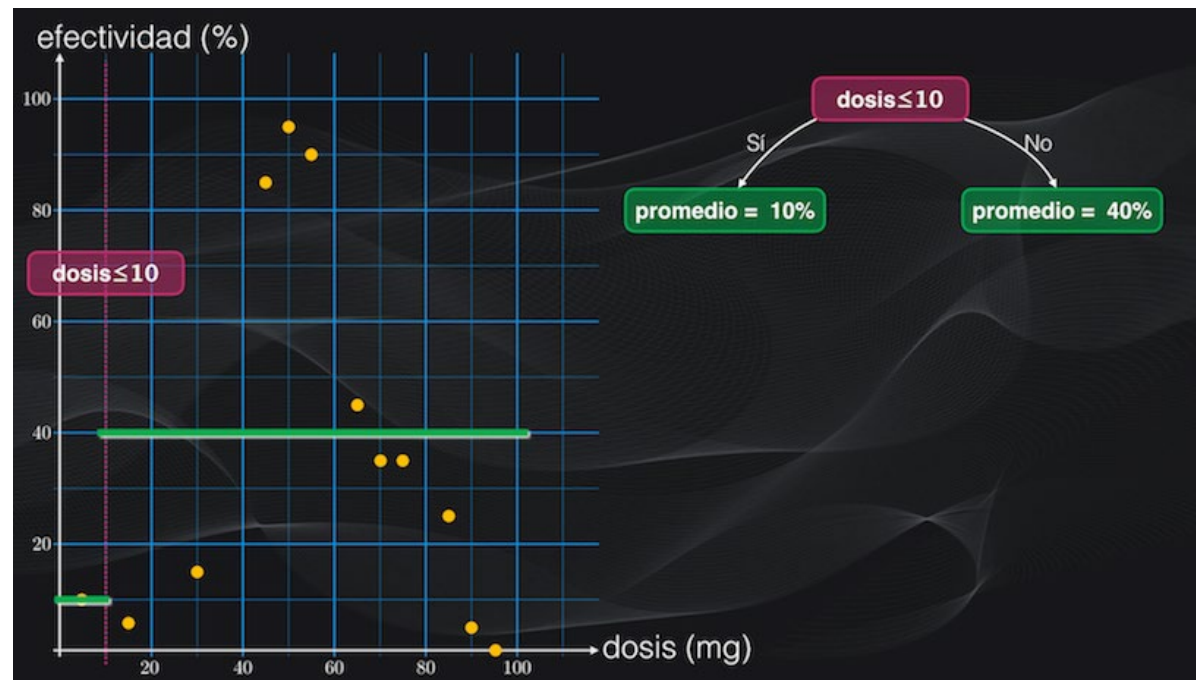
El primer paso es organizar ascendentemente los valores de la característica, y luego calcular el punto medio entre cada par de características consecutivas. A estos puntos los llamaremos “umbrales candidatos”.



Árbol de decisión: Regresión y CART

La idea ahora es seleccionar el mejor de estos “umbrales candidatos”, i.e, el que genere las particiones con la menor dispersión posible de la variable continua (que en este caso es la efectividad).

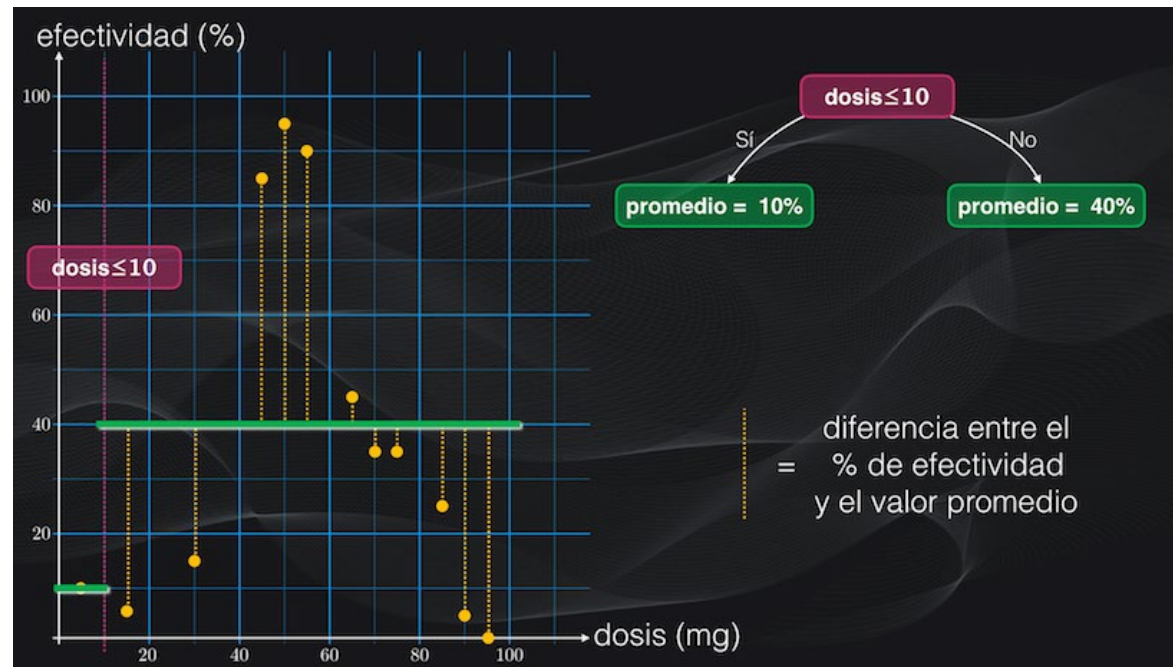
Para entender cómo medir esta dispersión analicemos el primer umbral: una dosis menor o igual a 10 mgr. Al usar esta condición la región izquierda tendrá un sólo punto y su dispersión será nula, porque su promedio será exactamente igual al valor de la efectividad, 10%.



Árbol de decisión: Regresión y CART

Pero la región derecha tiene un comportamiento diferente, pues tendrá 11 puntos con una efectividad promedio del 40%. Sin embargo, la mayoría de estos puntos se encuentra alejada del promedio, así que habrá una alta dispersión.

Para medir esta dispersión podemos simplemente promediar la diferencia existente entre el porcentaje de efectividad de cada punto y el valor de efectividad promedio de la región



Árbol de decisión: Regresión y CART

Pero para evitar que algunas diferencias positivas se anulen con diferencias negativas, elevaremos al cuadrado estas diferencias y luego sí las promediaremos. Y esta métrica que acabamos de definir se conoce como el error cuadrático medio:

$$ECM = \frac{1}{N} \sum_{i=1}^N (dato_i - promedio_{region})^2$$

donde N representa el número total de datos de la región, $dato_i$ es el porcentaje de efectividad de cada uno de los datos que hace parte de esta región y $promedio_{region}$ es el promedio de los porcentajes de efectividad de los datos que hacen parte de esta región.

Un error cuadrático medio igual a cero es la condición ideal, dispersión nula, mientras que entre más alto sea este error mayor será el grado de dispersión.

Si volvemos a las dos regiones que acabamos de obtener y calculamos su error cuadrático medio, veremos que la de la izquierda tiene un error igual a 0

$$ECM_{izq} = \frac{(10 - 10)^2}{1} = 0$$

Árbol de decisión: Regresión y CART

Los datos a la derecha tiene una dispersión bastante alta, pues su error es mucho mayor (1127.3)

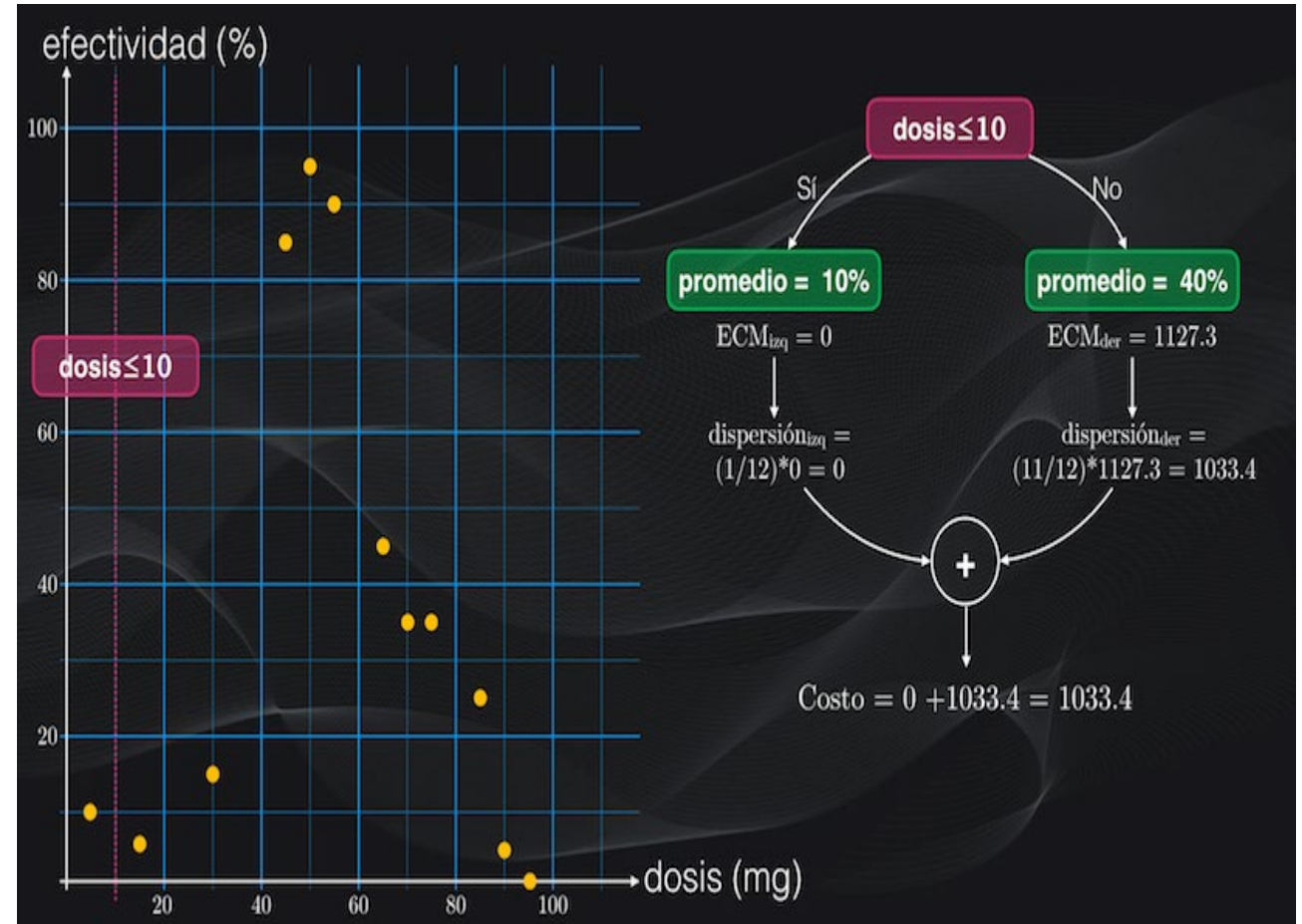
$$ECM_{der} = \frac{1}{11} \sum_{i=1}^{11} (dato_i - 40)^2$$

$$ECM_{der} = \frac{(5 - 40)^2 + (15 - 40)^2 + \dots + (0 - 40)^2}{11} = 1127.3$$

Ahora, sabiendo el grado de dispersión de cada subregión, podemos calcular un puntaje para el umbral seleccionado, lo que llamaremos en adelante función de costo.

Para esto primero calculamos la dispersión promedio de cada subregión: tomamos su error cuadrático medio y lo multiplicamos por la fracción de datos que resulta en cada agrupación, con relación al número inicial de datos.

Y con esta información calculamos la función de costo para este umbral, que será simplemente la suma de las dos dispersiones promedio (1033.4).

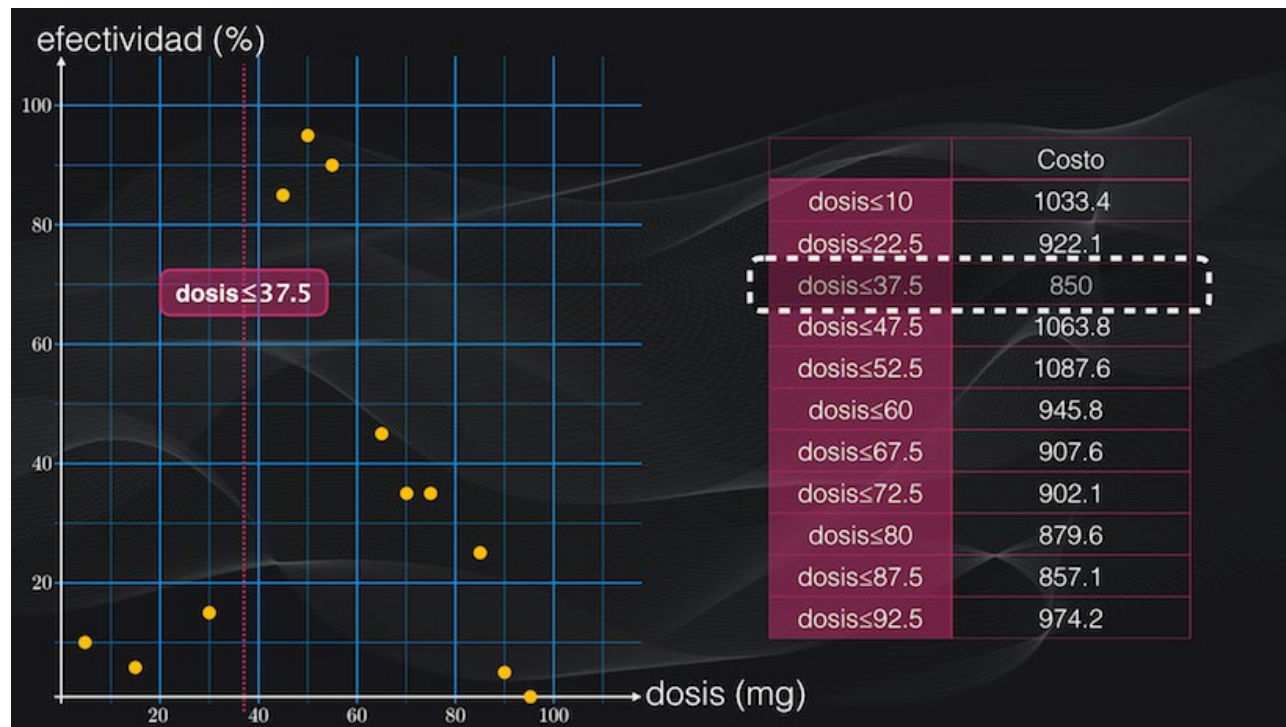


Árbol de decisión: Regresión y CART

Perfecto, ya tenemos una métrica para evaluar qué tan buena o no es una partición.

La idea es ahora repetir el mismo procedimiento del cálculo de la función de costo para cada umbral y una vez hecho esto tomar el umbral con la menor función de costo posible, que será precisamente el que genere las particiones con la menor dispersión.

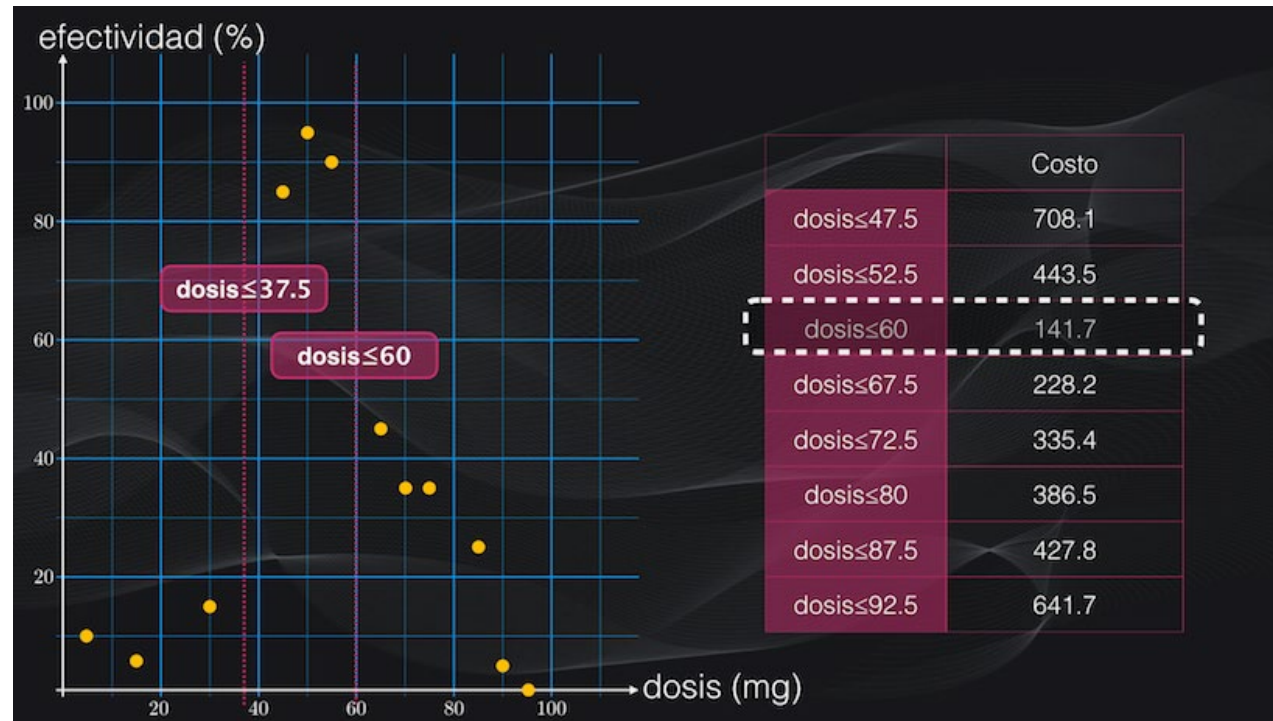
Aquí vemos que la mejor condición de todas es una dosis menor o igual a 37.5 mgr.



Árbol de decisión: Regresión y CART

Ahora lo que podemos hacer es seguir refinando cada una de las regiones obtenidas, realizando precisamente más particiones.

Si por ejemplo nos enfocamos en la región del lado derecho (derecha del umbral igual a 37.5 mg), vemos que aún se puede subdividir repitiendo el paso anterior.



Métricas en Regresión

Error absoluto medio (mean absolute error - MAE)

Mide la diferencia entre dos valores, i.e., nos permite saber que tan diferente es el valor predicho y el valor real. Para que un error con valor positivo no cancele a un error con error negativo usamos el valor absoluto de la diferencia.

Error medio cuadrado (mean square error - MSE)

Esta métrica es muy útil para saber que tan cerca es la línea de ajuste de nuestra regresión a las observaciones. Al igual que en caso anterior evitamos que un error con valor positivo anule a uno con valor negativo, pero en lugar de usar el valor absoluto, elevamos al cuadrado la diferencia.

Raíz del error medio cuadrado (root mean square error - RMSE)

Como la métrica anterior nos da el resultado en unidades cuadradas, para poder interpretarlo más fácilmente sacamos la raíz cuadrada y de esta manera tenemos el valor en las unidades originales.

Porcentaje de error medio absoluto (mean absolute percentage error - MAPE)

Esta es una métrica de la precisión, se expresa como un porcentaje, es independiente de la escala y fácil de interpretar. Como desventaja es que puede producir valores no definidos, infinito o muy cercanos a cero.

Métricas en Regresión

R² (R square)

Es el coeficiente de determinación, nos indica que tanta variación tiene la variable dependiente que se puede predecir desde la variable independiente. En otras palabras que tan bien se ajusta el modelo a las observaciones reales que tenemos. Cuando usamos R^2 todas las variables independientes que estén en nuestro modelo contribuyen a su valor.

El mejor valor posible que tenemos con R^2 es 1 y el peor es 0. Una desventaja que tiene es que asume que cada variable ayuda a explicar la variación en la predicción, lo cual no siempre es cierto. Si adicionamos otra variable, el valor de R^2 se incrementa o permanece igual, pero nunca disminuye. Esto puede hacernos creer que el modelo esta mejorando, pero no necesariamente es así.

R² Ajustado

Compensa la desventaja de R^2 con la adición de variables mediante la penalización la adición de variables independientes al modelo.

Error Logarítmico RMS (RMSLE)

Mide la proporción entre la predicción y el actual. Como RMSE es sensible a los outliers y esto nos puede llevar a que el valor del error se incremente mucho. Al usar los logaritmos, los outliers se ven escalados por lo que evitamos ese efecto. Cuanto más cercano sea el valor a 0, es mejor.

Métricas en Regresión

$$MAE = \frac{\sum_{i=1}^n |y_i - x_i|}{n}$$

$$MSE = \frac{\sum_{i=1}^n (y_i - x_i)^2}{n}$$

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - x_i}{y_i} \right|$$

$$R^2 = 1 - \frac{\sum (y_i - x_i)^2}{\sum (y_i - \mu_y)^2}$$

$$R^2 \text{ Ajustado} = 1 - \frac{(1 - R^2)(N - 1)}{N - M - 1}$$

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (y_i - x_i)^2}{n}}$$

$$RMSLE = \sqrt{\frac{\sum_{i=1}^n (\log(y_i + 1) - \log(x_i + 1))^2}{n}}$$

Métricas en Regresión

MSE y RMSE penalizan los errores grandes en la predicción, pero RMSE se usa más gracias a que esta en las unidades originales de los datos.

MSE/RMSE es una función diferenciable lo cual facilita ciertas operaciones matemáticas, en comparación a MAE que no es diferenciable. Muchos modelos usan como métrica por default RMSE para calcular la función de pérdida.

MAE es más robusto cuando los datos tienen outliers o datos atípicos y es la mejor opción a usar en esos casos.

Valores pequeños de MAE, MSE y RMSE nos indican mayor precisión en el modelo de regresión, pero hay que recordar que para R^2 un valor más grande es mejor.

R^2 y R^2 ajustado se usan para explicar que tan bien la variable independiente en la regresión lineal explica la variabilidad de la variable dependiente. R^2 siempre aumenta de valor cuando aumenta la cantidad de variables independientes en nuestro modelo.

R^2 ajustado toma en cuenta el número de variables independientes y su valor baja si el incremento en R^2 debido a las variables adicionales no es lo suficientemente significativo.

Para comparar la precisión entre diferentes modelos de regresión lineal generalmente es mejor usar RMSE que R^2 .

Estandarización vs Separación de datos

La estandarización de los datos debe realizarse **antes de entrenar** el modelo pero **después de separar** los datos en conjuntos de entrenamiento y prueba.

Esto es crucial por las siguientes razones:

1. Si estandarizas toda la base de datos antes de dividirla, estarías filtrando información de los datos de prueba hacia los de entrenamiento, lo que provocaría una "fuga de datos" (data leakage).
2. La estandarización debe calcularse únicamente con los datos de entrenamiento (media y desviación estándar) y luego aplicar esos mismos parámetros a los datos de prueba.

El proceso correcto sería:

1. Separar los datos en conjuntos de entrenamiento y prueba
2. Calcular los parámetros de estandarización (media y desviación estándar) usando solo los datos de entrenamiento
3. Aplicar esos mismos parámetros para estandarizar tanto los datos de entrenamiento como los de prueba
4. Entrenar el modelo con los datos de entrenamiento estandarizados
5. Evaluar el modelo con los datos de prueba estandarizados

Este enfoque garantiza que tu modelo sea evaluado en condiciones similares a las que enfrentaría en producción, donde no tendrías acceso a los datos futuros para calcular parámetros de estandarización.

Ejemplo en python

Conjunto de datos: *Conjunto de datos sobre diabetes*

Información del conjunto de datos:

Se obtuvieron diez variables basales, edad, sexo, índice de masa corporal, presión arterial promedio y seis mediciones de suero sanguíneo para cada uno de 442 pacientes con diabetes, así como la respuesta de interés, una medida cuantitativa de la progresión de la enfermedad un año después del inicio.

Información de atributos:

Las primeras 10 columnas (Age%, Sex%, Body mass index%, Average blood pressure%, S1%, S2%, S3%, S4%, S5% y S6) son valores numéricos y la columna 11 es una medida cuantitativa de la progresión de la enfermedad un año después del inicio.

Nota: Cada una de estas 10 características se ha centrado en la media y se ha escalado según la desviación estándar multiplicada por `n\_muestras`.

Ejemplo en python

```
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeRegressor, export_graphviz
from sklearn import metrics
from sklearn.model_selection import train_test_split
from graphviz import Source

""" Decision Tree - Regression """

dataDiabetes = pd.read_csv("diabetes.csv")

xDiabetes = dataDiabetes.drop('value', axis=1)
yDiabetes = dataDiabetes['value']

xDiabetesTrain, xDiabetesTest, yDiabetesTrain, yDiabetesTestReal = train_test_split(xDiabetes, yDiabetes, test_size=0.2)

tree_reg = DecisionTreeRegressor(max_depth=5, criterion='squared_error', random_state=13,
                                  min_samples_leaf=1, min_samples_split=2)

tree_reg.fit(xDiabetesTrain, yDiabetesTrain)

yDiabetesPred = tree_reg.predict(xDiabetesTest)
```

Ejemplo en python

```
print('Mean Absolute Error:', metrics.mean_absolute_error(yDiabetestReal, yDiabetesPred))  —————→ 48.117
print('Mean Squared Error:', metrics.mean_squared_error(yDiabetestReal, yDiabetesPred))  —————→ 3613.840
print('Root Mean Squared Error:', np.sqrt(metrics.mean_squared_error(yDiabetestReal, yDiabetesPred)))  —————→ 60.115
```

```
tableResult = pd.DataFrame({'Actual':yDiabetestReal, 'Predicted':yDiabetesPred})
```

	Actual	Predicted
391	63.0	93.772
339	95.0	190.400
98	92.0	132.487
19	168.0	132.487
81	51.0	132.487

Ejemplo en python

```
export_graphviz(tree_reg, out_file="diabetes.dot", feature_names=xDiabetes.columns, class_names=yDiabetes, rounded=True, filled=True)
```

```
Source.from_file("diabetes.dot")
```

